ScamGuard: AI-Based Online Scam and Fraud Detection System



PROJECT REPORT

Major Project Phase-2 (23CS7PWPP2), Semester VII

Submitted by

Bhavya Goyal (1BM23CS063)
Priyam Thakur (1BM23CS252)
Samir Chaudhary (1BM23CS294)
Sana Subodh (1BM23CS296)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of
Dr. Kayarvizhy N
Professor, Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU - 560019

Academic Year - 2026–2027

Date of Submission: November 2026

Certificate

This is to certify that the project report entitled **"ScamGuard: AI-Based Online Scam and Fraud Detection System"** is a bonafide work carried out by **Bhavya Goyal (USN: 1BM23CS063)**, **Priyam Thakur (USN: 1BM23CS252)**, **Samir Chaudhary (USN: 1BM23CS294)** and **Sana Subodh (USN: 1BM23CS296)** as Major Project Phase-2 (Course Code: 23CS7PWPP2) in the seventh semester, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science and Engineering**, under my supervision during the academic year 2026–2027.

Guide Signature	HoD signature	Principal Signature
(Dr. Kayarvizhy N)	Dr. Kavitha Sooda	Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

Abstract

Fraudulent and unwanted SMS messages are one of the cheapest ways to reach a phone user in India. They imitate bank KYC reminders, electricity disconnection notices, parcel updates and job offers, and they increasingly arrive in Hindi or in a mix of Hindi and English. ScamGuard is an AI-based system that classifies a message as scam or genuine, explains the verdict at phrase level and tells the user what to do next.

The core detector is trained on ScamGuard-IN-2026, a corpus of 11,399 Indian messages built from user-reported smishing, real Indian SMS, 2025–26 advisory texts and synthetic messages used for training only. Reported messages were audited: 19.5% of retained reports were genuine bank alerts, OTP messages or advertisements. Near-duplicate campaigns were grouped with MinHash and kept on one side of the train–test split. A logistic-regression classifier over word and character TF-IDF features, combined with 31 interpretable intent patterns, reaches 97.1% accuracy with a 3.2% false-alarm rate on 1,874 held-out real messages from unseen campaigns; training on unverified report labels raises the false-alarm rate to 34.0%.

As the advancement suggested by our guide, we studied cross-domain generalization and lightweight domain adaptation. Multinomial Naive Bayes and Logistic Regression trained on the UCI SMS Spam Collection (UK) were tested, without retraining, on an Indian SMS dataset. Logistic Regression fell from a spam F1-score of 0.911 to 0.678 and missed about half of the Indian spam. Adding 10% of a labelled Indian adaptation pool (143 messages) raised the mean target F1 over three seeds from 0.681 to 0.964, without lowering source-domain performance; the gain held in every seed, was significant by McNemar's test, and persisted when near-duplicate messages were kept apart. The system is deployed as a web application (Next.js front end, FastAPI back end, PostgreSQL), with a dedicated cross-domain page and a Streamlit application, and every number in this report is reproducible from the project repository.

Keywords: scam detection, SMS spam, smishing, domain shift, domain adaptation, TF-IDF, logistic regression, Naive Bayes, label noise, data leakage

DECLARATION

We hereby declare that the Major Project Phase-2 work entitled "ScamGuard: AI-Based Online Scam and Fraud Detection System" is a bonafide work and has been carried out by us under the guidance of Dr. Kayarvizhy N, Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	Bhavya Goyal	1BM23CS063	
2	Priyam Thakur	1BM23CS252	
3	Samir Chaudhary	1BM23CS294	
4	Sana Subodh	1BM23CS296	

Place: Bengaluru
Date:

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work titled "ScamGuard: AI-Based Online Scam and Fraud Detection System" as Major Project Phase-2 work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The work is original and we duly certify the same.

Guide Name: Dr. Kayarvizhy N
Date:
Signature

Acknowledgment

We are grateful to our guide, Dr. Kayarvizhy N, Professor, Department of Computer Science and Engineering, for steering this project towards questions that could be answered with evidence: first, a dataset that reflects the messages people in India actually receive, and then the cross-domain generalization and adaptation study that she proposed as the advancement of this work.

We thank Dr. Kavitha Sooda, Head of the Department of Computer Science and Engineering, and Dr. Bheemsha Arya, Principal, B.M.S. College of Engineering, for the facilities and the academic environment that made this work possible. We also thank the faculty members who reviewed our phase presentations and pointed out gaps in our evaluation.

We acknowledge the researchers who released the IMC 2025 smishing collection, the UCI SMS Spam Collection and the India Spam SMS collection under open licences. Finally, we thank our families and friends, several of whom forwarded the suspicious messages that shaped our test cases.

Contents

1 Introduction **1**
1.1 Background of the Study 1
1.2 Research Motivation 1
1.3 Objective 2
1.4 Scope 2
1.5 SDG Justification 3
1.6 Existing System 3
1.7 Proposed System 4
1.8 Work Plan 4

2 Literature Survey **6**
2.1 Overview 6
2.2 Detailed Survey 6
2.3 Research gaps 8
2.4 Mapping of Project Objectives with Research Gaps 9

3 Requirement Analysis and Specification **10**
3.1 Stakeholders 10
3.2 Functional Requirements 10
3.3 Non-Functional requirements 11
3.4 System Requirements 12
3.5 Data Requirements 13
3.6 Constraints and Limitations 13
3.7 Assumptions 13
3.8 Risks and Challenges 14

4 Designs **15**
4.1 System Architecture Overview 15
4.2 Module/Component Design 16
4.3 Detailed Design 17
4.3.1 Client–server communication 17
4.3.2 Dataset construction 17

4.3.3 Cross-domain experiment design . . . 18
4.3.4 Inference procedure . . . 19
4.4 Data Design . . . 19
4.4.1 Database . . . 19
4.4.2 Datasets . . . 20

5 Methodology **21**
5.1 Tools and Techniques . . . 21
5.2 Data Set with Explanation . . . 22
5.2.1 ScamGuard-IN-2026 (main detector) . . . 22
5.2.2 Datasets of the cross-domain study . . . 23
5.3 Flowchart of Methodology from Start to End . . . 24
5.4 Explanation of Methodology in detail with Equations and Formulae . . . 26
5.4.1 Problem definition . . . 26
5.4.2 Preprocessing . . . 27
5.4.3 Feature extraction . . . 27
5.4.4 Logistic regression . . . 28
5.4.5 Multinomial Naive Bayes . . . 28
5.4.6 Intent evidence and fusion . . . 28
5.4.7 Near-duplicate grouping . . . 29
5.4.8 Domain adaptation . . . 29
5.4.9 Evaluation metrics . . . 29
5.4.10 Statistical test . . . 29

6 Results and Discussion **31**
6.1 Presentation of Results . . . 31
6.1.1 Experimental setup and testing . . . 31
6.1.2 Main detector on ScamGuard-IN-2026 . . . 31
6.1.3 Cross-domain generalization (RQ1 and RQ2) . . . 34
6.1.4 Lightweight domain adaptation (RQ4) . . . 35
6.1.5 Word versus character features (RQ3) . . . 37
6.1.6 Error analysis (RQ5) . . . 38
6.1.7 Application . . . 39
6.2 Limitations of the study . . . 39
6.3 Conclusion . . . 39

7 Conclusion **41**

References **42**

Plagiarism **45**
Appendix A: Snapshots 46
Appendix B: Details of list of publications related to this project 52
Appendix C: Details of Patent 53
Appendix D: Details of Funding 54
Appendix E: POs and PSOs Mapped 55
Appendix F: Similarity report, AI Generated report 58

List of Figures

4.1 Architecture of the deployed system. Shaded parts run offline. 15
4.2 Sequence diagram of one message analysis 17

5.1 Class distribution of Dataset A (UCI) and Dataset B (India) before cleaning 24
5.2 Message length distribution of the two datasets 24
5.3 Flow of the methodology 25
5.4 Flow of the cross-domain study 26

6.1 Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%) 32
6.2 False-alarm rate with report labels and with audited labels 33
6.3 ROC and precision–recall curves on the real test set 34
6.4 (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold 34
6.5 Spam F1 on the final Indian test set as labelled Indian messages are added (mean ± SD over three seeds) 36
6.6 Logistic Regression (word TF-IDF, seed 42) on the Indian test set, before and after adding 40% of the pool 36
6.7 Target F1 of the source-only models with word and character TF-IDF . . 37
6.8 Error categories before and after adaptation 38

A.1 Home page with the quick check 46
A.2 Quick check of a fake KYC message: verdict, matched tactics, known pattern and Scam X-Ray 46
A.3 Quick check of a genuine operator notice 47
A.4 Full analysis of a scam message for a signed-in user 47
A.5 Scam X-Ray: each risky phrase placed in the hook, pressure, request and cash-out stages 48
A.6 TrustLens: scam journey, recommended actions and how a genuine message differs 48
A.7 Dashboard with priority findings and recent scans 49
A.8 Scan history with search, filters and export 49
A.9 Analytics: verdict share, distribution and score trend 50

A.10 Safety tools: QR code check, before-you-pay, file check and chat timeline 50
A.11 Cross-domain study page: the adapted model flags an Indian promotion that the baseline misses, with the measured results 51

List of Tables

1.1 Work plan 5

2.1 Summary of related work 7
2.2 Mapping of objectives to research gaps 9

3.1 Stakeholders and their needs 10
3.2 Hardware requirements 12
3.3 Software requirements 12
3.4 Risks and how they are handled 14

4.1 Modules of ScamGuard and their interfaces 16
4.2 Main database tables 20

5.1 Sources of ScamGuard-IN-2026 22
5.2 ScamGuard-IN-2026 splits 23
5.3 Cross-domain datasets before and after cleaning 23

6.1 Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate) 31
6.2 Classifiers on the real test set (%); “Challenge” is F1 on the 259 team-written messages 32
6.3 Effect of the training data (logistic regression, word + char, %) 33
6.4 Trained on Dataset A only (seed 42): spam F1 on the UCI test split and on all of Dataset B 35
6.5 Spam F1 on the final Indian test set (615 messages), word TF-IDF, mean ± SD over three seeds 36
6.6 Target F1 with a random and a group-aware split of Dataset B (word TF-IDF, mean ± SD) 37
6.7 Source-only models: word versus character TF-IDF (mean ± SD over three seeds) 37
6.8 Error categories on the Indian test set (Logistic Regression, word TF-IDF, seed 42) 38

E.1 Course outcomes and where they are addressed 55

E.2 CO–PO–PSO mapping of Major Project Phase-2 55
E.3 Programme Outcomes Mapping with Project 56
E.4 Programme Specific Outcomes Mapping with Project 57

Chapter 1

Introduction

1.1 Background of the Study

A scam SMS rarely looks like a scam. It looks like a message from a bank asking for a quick KYC update, from an electricity board warning of disconnection tonight, from a courier holding a parcel for a small fee, or from the traffic police with a challan to pay through an attached app. Fraudsters copy the format of genuine service messages because people have learned to trust those formats. A detector has to tell the two apart even though they share most of their vocabulary: both mention OTPs, accounts, KYC, links and deadlines.

Machine-learning filters for SMS have a long history, but most published results train and test on random splits of a single collection, most often the UCI SMS Spam Collection [4], and report accuracies above 95% [5, 6]. Those numbers describe messages that resemble the training data. A filter deployed in India meets different brands, currencies, link styles and languages, so its real accuracy can be much lower.

ScamGuard is a web application that classifies a single message pasted by the user. It returns a verdict, a risk score and the reasons: the phrases that make the message risky, the stage of the scam each phrase belongs to, what a genuine message from the same organisation would do instead, and what the user should do next. Behind it are a text classifier, a layer of interpretable rules, an audited Indian dataset, and a cross-domain study of how such classifiers behave when the data changes.

1.2 Research Motivation

Four observations motivated the project.

First, the scale and the Indian focus of the problem. A 2025 measurement study of more than 64,000 public smishing reports found that most attributable sender numbers belonged to Indian mobile operators [1].

Second, the cost of false alarms. Genuine Indian bank alerts, OTP messages and operator notices use the same words as scams. Public smishing data contains only

reported messages, and many reports are themselves genuine. A tool that flags real bank messages teaches users to ignore warnings.

Third, the lack of explanation. A bare "spam" label gives the user nothing to act on. Users need to see which part of the message is the trap.

Fourth, domain shift. A model trained on one country's messages may fail on another's. Learning theory bounds the error on a new domain by the source error plus the difference between domains [21], and a small labelled sample from the new domain is a common, cheap remedy [22, 23]. Whether this holds for SMS spam, and how much local data is needed, had not been measured for Indian messages.

1.3 Objective

The objectives of the project are:

1. To build a labelled dataset of Indian scam and genuine SMS that reflects current scam types and whose labels have been checked, rather than taken from user reports at face value.
2. To evaluate lightweight text classifiers on that dataset under a protocol that prevents near-duplicate messages from leaking between training and testing.
3. To combine the best classifier with interpretable intent rules that explain each verdict at phrase level, and to measure honestly whether the rules also improve accuracy.
4. To measure the cross-domain generalization of conventional SMS spam classifiers (Naive Bayes and Logistic Regression with word and character TF-IDF) trained on the UCI SMS Spam Collection and tested on Indian SMS, and to study lightweight supervised domain adaptation with 10%, 20% and 40% of labelled target data.
5. To deliver the detector and the cross-domain models in a usable, deployed web application with supporting safety tools, history and analytics.
6. To make every reported result reproducible from the project repository.

1.4 Scope

The system analyses one message at a time, as pasted by the user, in English, Hindi (Devanagari script) or romanised Hindi. The main detector classifies a message as *scam* or *legitimate*; a scam tries to obtain money, a secret such as an OTP or PIN, personal data, an app installation or a call to a private number by deception, and genuine advertisements are legitimate. The cross-domain study uses the broader *spam* versus *ham* labels of its

two public datasets, where spam includes unwanted promotions. The application also accepts URLs, e-mails, screenshots, PDFs and QR codes, which it converts to text or checks with separate tools.

The project does not intercept messages on the phone, does not use sender metadata or conversation history in the classifier, and does not cover Indian languages other than Hindi. The cross-domain study uses two datasets and does not claim robustness across domains in general. The risk score is a ranking signal, not a calibrated probability, and the verdict is advice, not proof of fraud.

1.5 SDG Justification

The project addresses the following United Nations Sustainable Development Goals.

- **SDG 16 (Peace, Justice and Strong Institutions), target 16.4 on reducing illicit financial flows.** Scam messages are the entry point of many financial frauds. Helping users recognise them before they pay or share an OTP reduces the money that reaches fraudsters, and the application points users to the official reporting channels (helpline 1930 and cybercrime.gov.in).
- **SDG 8 (Decent Work and Economic Growth), target 8.10 on access to financial services.** Digital payments widen financial access only if people can trust them. A tool that separates genuine bank and UPI messages from imitations supports that trust.
- **SDG 9 (Industry, Innovation and Infrastructure), target 9.c on access to ICT.** The detector is small enough to run on free cloud infrastructure and answers in milliseconds, so it can be offered at no cost. The cross-domain study shows that a model can be adapted to a new region with only a few hundred labelled local messages.
- **SDG 10 (Reduced Inequalities).** Support for Hindi messages and a simplified elder mode aim at users who are most often targeted and least served by English-only tools.

1.6 Existing System

Existing approaches fall into three groups.

- **Operator and regulator measures.** Telecom operators filter bulk traffic and register commercial senders, and users can report messages to the 1909 DND service or the cyber-crime portal. These act on senders and volumes; they do not

help a user judge a message that has already arrived.

- **Rule-based and keyword filters.** Phone apps flag words such as "lottery" or "KYC". They are easy to explain but flag genuine messages that use the same words and miss scams that avoid them.
- **Machine-learning filters in research.** Naive Bayes, SVM, CNN and LSTM classifiers report high accuracy, but mostly on promotional spam, with random train–test splits within one dataset, without a genuine Indian class and without testing on messages from another region [5, 6, 9]. Newer report-based corpora contain only reported messages [10, 1].

1.7 Proposed System

The proposed system has four parts.

1. **A dataset, ScamGuard-IN-2026.** Real Indian reported scams, real Indian genuine messages, 2025–26 advisory texts and synthetic messages for the newest scam types, with every reported-message group checked and the data split by scam campaign.
2. **A detector.** Logistic regression over word and character TF-IDF features combined with interpretable intent rules and genuine-message guards that explain each verdict.
3. **A cross-domain study.** Naive Bayes and Logistic Regression trained on the UCI SMS Spam Collection and evaluated on Indian SMS before and after adding 10%, 20% and 40% of a labelled Indian adaptation pool, repeated with three seeds, with paired significance tests, a near-duplicate check and a manual error analysis.
4. **A complete application.** A deployed web application with a quick check, a full analysis with Scam X-Ray and TrustLens explanations, an AI copilot, safety tools, history, analytics, a cross-domain page that compares the baseline and adapted models, and a separate Streamlit application for the study.

1.8 Work Plan

Table 1.1 shows the work plan across the two project phases.

Table 1.1: Work plan

Phase	**Activity**	**Outcome**
1	Problem study and literature survey	Research gaps: no verified Indian genuine class, leakage-prone and single-domain evaluation
1	Requirement analysis and design	System architecture, data model and user flows
1	Application architecture	FastAPI back end, Next.js front end, model registry, database
2	Explanation layer	Intent rules, Scam X-Ray, TrustLens, retrieval of known patterns
2	Dataset construction	ScamGuard-IN-2026 with label audit and campaign split
2	Model selection and experiments	Final model; leakage, label-audit, training-data and fusion experiments
2	Cross-domain study (guide's advancement)	UCI to Indian SMS evaluation, 10/20/40% adaptation over three seeds, error analysis, Streamlit app and web page
2	Deployment and testing	Vercel, Render and Neon deployment; 836 back-end and 14 front-end automated tests
2	Documentation	Report, research paper, presentation

Chapter 2

Literature Survey

2.1 Overview

We reviewed nineteen works on SMS spam corpora, smishing detection, report-based measurement studies, domain shift and adaptation, and machine-learning methodology for label noise and data leakage. Each reference was checked against its publisher page, DOI record or arXiv entry. Results quoted are those stated by the authors.

2.2 Detailed Survey

SMS Spam Collection. Almeida, Gómez Hidalgo and Yamakami released the SMS Spam Collection in 2011 [4]: 5,574 English messages, 747 of them spam, mostly from the UK. In a follow-up they compared many classifiers and preferred a linear SVM, which reached 97.64% accuracy while blocking 0.18% of legitimate messages [5]. Its positive class consists of prize draws and ringtone offers rather than current Indian scams.

Deep learning on the same corpus. Roy, Singh and Banerjee trained CNN and LSTM models and reported up to 99.44% accuracy [6]. Ahmadi *et al.* compared six classifiers with TF-IDF and bag-of-words features and reported accuracies between 0.91 and 0.95 [13]. Labonne and Moran showed that a fine-tuned Flan-T5 model beats classical baselines for e-mail spam with little training data [12]. All of these evaluate within one dataset.

Smishing detection. Sonowal and Kuppusamy proposed SmiDCA, which selects content features for anti-smishing classifiers [8]. Mishra and Soni combined content analysis with the behaviour of embedded URLs and reported 96.29% accuracy [7]. Goel *et al.* trained Naive Bayes on 362 smishing messages with slang normalisation and reached 96.2% accuracy with a 3.87% false-positive rate [9].

Report-based corpora. Timko and Rahman released 1,062 validated Smishtank submissions [10]. Agarwal *et al.* released 33,869 user-reported messages in 66 languages, labelled with GPT-4o, and state that reports carry no verification [1]. The same group found that 35.12% of unique UK operator reports were spam rather than

scams [2]. MOZ-Smishing provides genuine and fraudulent mobile-money messages for Mozambique [11].

Domain shift and adaptation. Ben-David *et al.* bound the error of a classifier on a target domain by its source error plus a measure of divergence between the domains [21]. Pan and Yang survey transfer learning, including the setting with a small amount of labelled target data [22]. Daumé III shows that simple ways of combining source and target data are strong baselines for supervised domain adaptation in NLP [23]. None of the SMS studies above tests a model on messages from a different collection.

Surveys and methodology. Pritom *et al.* reviewed 119 smishing papers and found datasets "often limited in scale, diversity, and temporal coverage" [3]. Northcutt, Jiang and Chuang showed that cross-validated disagreement locates label errors [14]. Lee *et al.* showed that near-duplicates inflate evaluation [15], and Kapoor and Narayanan list train–test duplication among the most common leakage errors [16]. MinHash [17] is the standard tool for finding near-duplicates.

Table 2.1 summarises the surveyed works.

Table 2.1: Summary of related work

No.	Work	Technique	Result	Limitation for this project
1	Almeida *et al.*, 2011 [4]	SMS spam corpus and baselines	5,574 labelled messages	UK promotional spam, no Indian messages
2	Almeida *et al.*, 2013 [5]	15+ classifiers, tokenisers	SVM 97.64% accuracy	Same corpus; single domain
3	Roy *et al.*, 2020 [6]	CNN, LSTM	99.44% accuracy	No cross-domain test
4	Sonowal, Kuppusamy, 2018 [8]	Feature selection, ML classifiers	Anti-smishing model	Small English datasets
5	Mishra, Soni, 2020 [7]	Content + URL behaviour	96.29% accuracy	Misses scams without links
6	Goel *et al.*, 2024 [9]	Slang normalisation, Naive Bayes	96.2% acc., FPR 3.87%	Only 362 smishing messages
7	Timko, Rahman, 2024 [10]	Validated report collection	1,062 reports	No genuine class

No.	Work	Technique	Result	Limitation for this project
8	Agarwal *et al.*, 2025 [1]	GPT-4o labelling of reports	33,869 messages	Reports unverified; no genuine class
9	Agarwal *et al.*, 2026 [2]	Analysis of operator reports	35.12% of reports are spam	UK data; no classifier
10	Ali *et al.*, 2025 [11]	LLM in-context learning	Mobile-money benchmark	Mozambique context
11	Labonne, Moran, 2023 [12]	Fine-tuned Flan-T5	Beats classical baselines	E-mail, not SMS
12	Ahmadi *et al.*, 2025 [13]	TF-IDF, six classifiers	0.91–0.95 accuracy	Single dataset
13	Pritom *et al.*, 2026 [3]	Systematic review	119 papers, 12 datasets	Identifies the dataset gap
14	Ben-David *et al.*, 2010 [21]	Theory of domain adaptation	Target-error bound	General theory
15	Pan, Yang, 2010 [22]	Transfer-learning survey	Taxonomy of settings	Not SMS-specific
16	Daumé III, 2007 [23]	Feature augmentation	Strong simple baseline	NLP tasks other than SMS
17	Northcutt *et al.*, 2021 [14]	Confident learning	Finds label errors	General method
18	Lee *et al.*, 2022 [15]	Near-duplicate removal	Overlap inflates results	Language-model corpora
19	Kapoor, Narayanan, 2023 [16]	Survey of leakage errors	Duplicates are common	General method

2.3 Research gaps

The survey points to five gaps.

1. **No verified genuine Indian class.** Classical corpora are not Indian; report-based corpora contain only reported messages.
2. **Report labels are taken at face value,** although their own authors warn that reports are unverified.

3. **Evaluation ignores campaigns.** Random splits put copies of one scam template on both sides, which inflates results.
4. **Rules are not evaluated independently.** Hybrid systems add rules, but measure their benefit on messages chosen by the same developers.
5. **No cross-domain evaluation for SMS spam.** Published SMS classifiers are trained and tested within one collection; how far they fall on another country's messages, and how much labelled local data restores them, is not measured.

The resulting problem statement is:

To design, implement and evaluate an AI-based system that classifies an Indian SMS as scam or genuine with a low false-alarm rate, using audited labels and a leakage-free split; to measure how conventional spam classifiers trained on one SMS domain generalize to Indian SMS and how much labelled target data adaptation needs; and to deliver both as a deployed web application that explains each verdict.

2.4 Mapping of Project Objectives with Research Gaps

Table 2.2: Mapping of objectives to research gaps

Objective	**Gap addressed**
1. Build an Indian dataset with checked labels	Gaps 1 and 2: genuine Indian class; audit of report labels
2. Evaluate under a leakage-free protocol	Gap 3: campaign-aware split
3. Combine classifier with interpretable rules and measure their value	Gap 4: rules evaluated on independent real data
4. Cross-domain generalization and domain adaptation	Gap 5: UCI-to-Indian evaluation and 10/20/40% adaptation
5. Deliver a deployed application	Practical use of the findings
6. Reproducibility	Supports verification of all five gaps

Chapter 3

Requirement Analysis and Specification

3.1 Stakeholders

Table 3.1: Stakeholders and their needs

Stakeholder	**Need**
Mobile phone users	Know quickly whether a message is a scam, why, and what to do next
Elderly and first-time digital users	A simple interface, Hindi support and clear next steps such as calling 1930
Families and guardians	Check a message forwarded by a relative and share a warning
Researchers and faculty	Reproducible data, protocol and results; a study of cross-domain behaviour
Administrators (project team)	Monitor model versions, metrics and the health of the deployed service
Banks, telecom operators and reporting agencies	Fewer users falling for impersonation; users directed to official channels

3.2 Functional Requirements

1. **Quick check.** A visitor can paste a message on the home page and receive a verdict, risk score, highlighted trap phrases and a short explanation without an account.
2. **Account management.** Users can register and sign in (with a captcha) or open a demo account, and update, export or delete their data.
3. **Full analysis.** A signed-in user can analyse text, a URL, an e-mail, a screenshot, a PDF or a QR code, and see the verdict, classifier probability, final risk score, Scam X-Ray stages, TrustLens report and recommended actions.

4. **Explanation.** Each result shows the matched phrases with their scam stage (hook, pressure, request, cash-out), evidence for and against, and how a genuine message would differ.
5. **Cross-domain study page.** A visitor can classify a message with the baseline (UCI-only) or adapted (UCI + 40% Indian) model, choose Naive Bayes or Logistic Regression, and see the label, spam probability, model version and the features behind the prediction, together with the study's measured results.
6. **AI copilot.** Users can ask follow-up questions in English or Hindi, answered by a hosted language model with per-visitor and daily limits and built-in fallback answers.
7. **Safety tools.** QR code check, a "before you pay" check for UPI requests, a payment-screenshot check, a file check for disguised Android apps and a chat timeline.
8. **History, cases and reports.** Users can search past scans, group them into cases, give feedback, download a PDF report and report a scam to the community view.
9. **Analytics and administration.** Users see totals and trends; administrators see registered model versions and run a robustness test.

3.3 Non-Functional requirements

- **Performance.** Classification of one message takes about 2 ms, and about 4 ms with the intent layer, on a two-core server; a full web request completes in about 1–2 seconds.
- **Reliability.** Models are loaded once at start-up from versioned files; the back end keeps itself awake on the free tier, and the front end retries transient proxy errors.
- **Security.** Passwords are hashed with bcrypt, access uses signed JWT tokens with refresh rotation, endpoints are rate-limited, request bodies are size-limited, outbound URL fetching is SSRF-safe, and security headers are set.
- **Privacy.** The quick check and the cross-domain page store nothing; contributed messages are masked before storage.
- **Usability.** English and Hindi interface, elder mode, keyboard-accessible controls and layouts that work at phone width.
- **Maintainability and reproducibility.** Training and serving share the same preprocessing code; 836 back-end and ML tests and 14 front-end tests run in continuous integration; one notebook and one script regenerate every reported number.

3.4 System Requirements

Table 3.2: Hardware requirements

Component	**Requirement**
Development machine	Any laptop with 8 GB RAM and a recent 64-bit CPU; no GPU is needed
Training	Under 3 seconds for the production model and about one minute for the full cross-domain notebook on a laptop CPU
Server	One small cloud instance; 512 MB RAM is sufficient
Client	Any browser on a phone or computer

Table 3.3: Software requirements

Layer	**Software**
Language	Python 3.10+ (tested on 3.13), TypeScript 5.5
Machine learning	scikit-learn 1.7.2, NumPy, pandas, SciPy, joblib, datasketch, ftfy, matplotlib
Notebook and study app	Jupyter or Google Colab, Streamlit
Back end	FastAPI 0.115, Uvicorn, SQLAlchemy 2.0, Alembic, Pydantic 2, SlowAPI, python-jose, passlib, ReportLab, Tesseract OCR
Front end	Next.js 14.2, React 18.3, Tailwind CSS 3.4, Recharts, Vitest
Database	PostgreSQL (Neon) in production, SQLite locally
Deployment	Vercel (front end), Render (back end), GitHub Actions (continuous integration)

3.5 Data Requirements

- **Main detector:** labelled Indian scam and genuine SMS with verified labels, a campaign identifier for each message and a split that keeps campaigns apart (ScamGuard-IN-2026, 11,399 messages).
- **Cross-domain study:** a labelled source dataset (UCI SMS Spam Collection, 5,574 messages) and an independently collected labelled target dataset (India Spam SMS Classification, 2,267 messages), both with documented licences, audited for missing values, duplicates, label conflicts, encoding and overlap.
- **Format:** plain text with a binary label (scam/legitimate or spam/ham, mapped to 1/0).
- **Personal data:** phone numbers, account numbers, OTPs and names masked before storage; no private messages collected without consent.

3.6 Constraints and Limitations

- Free-tier hosting limits memory to 512 MB, which rules out large transformer models in production.
- Messages are analysed one at a time without sender metadata or conversation history.
- Only English and Hindi are covered.
- Real 2025–26 Indian scam messages with verified labels are scarce; the 2026 test set has 12 messages.
- The cross-domain study uses only two datasets, and the Indian dataset has no documented labelling procedure.

3.7 Assumptions

- The user pastes the full text of one message.
- Audited labels are correct; unclear messages were dropped rather than guessed.
- The public datasets are representative enough of their domains for a controlled comparison.
- The final target test set is never used for training, feature fitting or choosing settings.

3.8 Risks and Challenges

Table 3.4: Risks and how they are handled

Risk	**Mitigation**
Wrong labels in reported data	Label audit by rule and by hand; second pass with cross-validated disagreement
Inflated results from near-duplicates	Campaign-aware split; group-aware split check in the cross-domain study
New scam types not in training data	Synthetic training messages; intent rules; small labelled target sample (adaptation)
Obfuscated spellings (K.Y.C, bl0cked)	Intent rules catch them; normalising such spellings before feature extraction is future work
Free-tier server sleeping	Keep-alive pings, front end on Vercel, automatic retries
External AI service slow or unavailable	Time limits and built-in fallback answers; the detector does not depend on it

Chapter 4

Designs

4.1 System Architecture Overview

ScamGuard is a three-tier web application. The **presentation tier** is a Next.js front end hosted on Vercel; it proxies all API calls through its own origin. The **application tier** is a FastAPI back end hosted on Render, which runs the detector, the intent rules, the cross-domain models, authentication and the safety tools in one process. The **data tier** is a PostgreSQL database on Neon that stores users, scans, cases and reports. Models are trained offline and stored as versioned files that the back end loads once at start-up. Figure 4.1 shows the components and their connections.

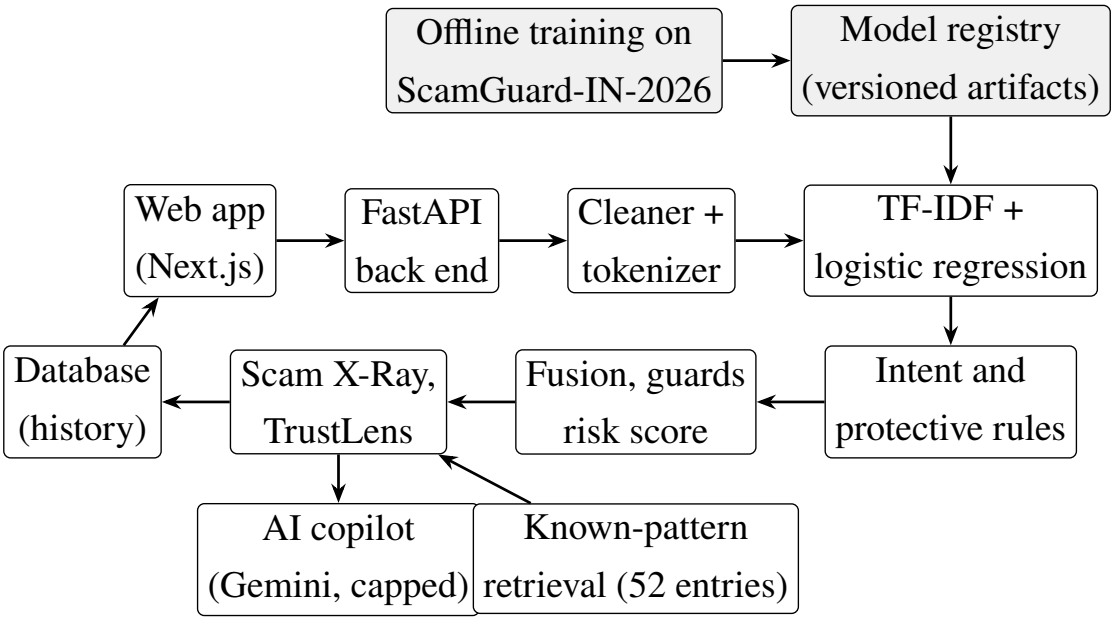


Figure 4.1: Architecture of the deployed system. Shaded parts run offline.

4.2 Module/Component Design

Table 4.1 lists the modules of ScamGuard, their responsibility and their interfaces.

Table 4.1: Modules of ScamGuard and their interfaces

Module	**Responsibility**	**Interface**
Dataset builder	Collect sources, audit labels, group campaigns, split	Python script; writes one CSV file
Training pipeline	Train and compare models; register the selected one	Model registry (vectorizer and model files)
Detector	Clean, classify, apply rules, fuse, explain	Python function predict(text)
Cross-domain study	Audit and clean UCI and Indian data, run the 7 experiments over 3 seeds, save results and models	Jupyter notebook; CSV tables, figures, results.json, joblib models
Cross-domain service	Serve the baseline and adapted models with explanations	POST /public/cross-domain/classify, GET /public/cross-domain/results
Back end (API)	Authentication, rate limits, storage, reports, safety tools	REST over HTTPS, JSON
Front end	Quick check, analysis, dashboard, tools, cross-domain page, Hindi	Browser; calls the REST API
Streamlit app	Stand-alone demonstration of the study	Loads the saved models; never retrains
Copilot	Answers follow-up questions	REST endpoint; external LLM API with limits

4.3 Detailed Design

4.3.1 Client–server communication

The web client and the back end communicate over HTTPS with JSON bodies. Figure 4.2 shows the sequence for one analysis.

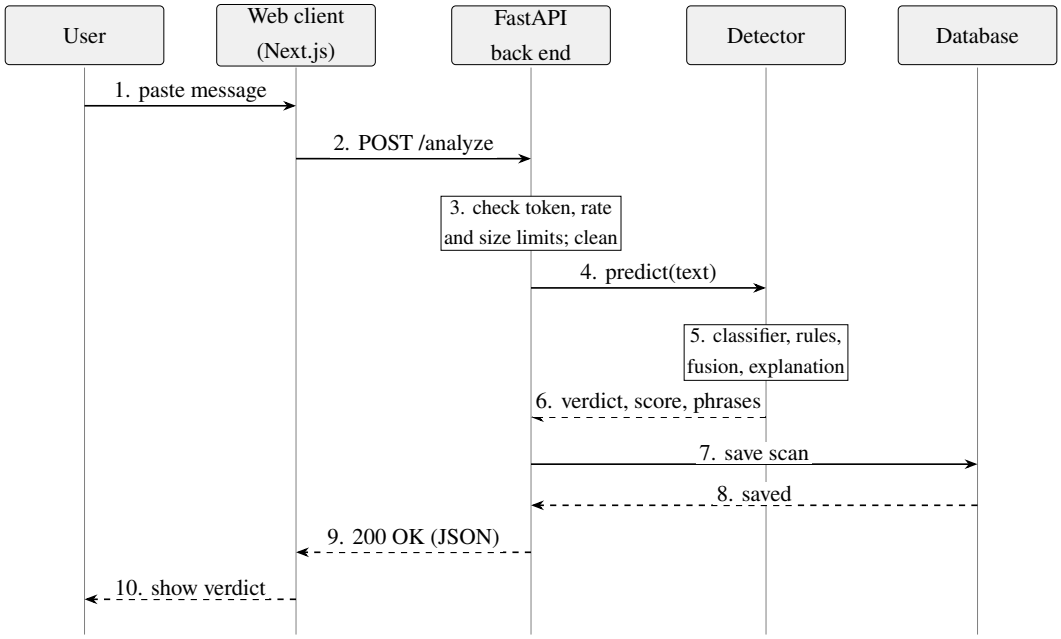


Figure 4.2: Sequence diagram of one message analysis

- **Endpoints.** POST /api/v1/public/quick-check serves visitors without an account (10 requests per minute); POST /api/v1/messages/analyze serves signed-in users (30 per minute); POST /api/v1/public/cross-domain/classify serves the cross-domain page (30 per minute). Other routes handle authentication, history, cases, analytics, the copilot and the safety tools.
- **Authentication.** Signed access tokens with refresh tokens; passwords stored as bcrypt hashes.
- **Robustness.** Message text is limited to 4,000 characters (2,000 for the public endpoints).

4.3.2 Dataset construction

Algorithm 1 summarises how ScamGuard-IN-2026 is built and split.

Algorithm 1 Building and splitting ScamGuard-IN-2026

Require: sources R1–R3, synthetic templates S, audit files

normalise anonymised entities in R1; mask personal data
group real messages into campaigns by MinHash ($J \geq 0.6$)
for each reported campaign **do**
    if strong fraud cue and no genuine cue **then**
        label scam
    else
        apply the manual decision (scam, genuine or drop)
    end if
end for
re-read rows that a cross-validated model strongly contradicts
assign each campaign to train, development or test by a salted hash
add synthetic messages to training only; remove exact test duplicates from training
return labelled dataset with split column

4.3.3 Cross-domain experiment design

Algorithm 2 gives the protocol of the cross-domain study. All choices are made on source data; the final target test set is used only for scoring.

Algorithm 2 Cross-domain evaluation and lightweight domain adaptation

Require: cleaned source set A (UCI), cleaned target set B (Indian), seeds $\{42, 123, 2026\}$, fractions $\{0, 0.1, 0.2, 0.4\}$

choose α (Naive Bayes) and C (Logistic Regression) by 5-fold cross-validation on A_{train}
for each seed **do**
    split A 80/20 into A_{train}, A_{test} and B 70/30 into pool P and test B_{test} (stratified)
    for each fraction f **do**
        $S_f \leftarrow$ stratified sample of $f \cdot |P|$ messages from P
        fit TF-IDF and the classifier on $A_{train} \cup S_f$
        score A_{test} and B_{test}; store predictions
    end for
end for
report mean ± SD over seeds; McNemar test of each adapted model against $f = 0$

4.3.4 Inference procedure

Algorithm 3 Scoring one message in the live service

Require: message x; frozen vectorizer and model; intent rules

$t \leftarrow \textsc{Tokenize}(\textsc{Clean}(x))$
$p \leftarrow \sigma(\mathbf{w}^\top \phi(t) + b)$
find matched intents I and protective signals P
if a sentence asks for an OTP after a "never share" warning **then**
    add decoy intent
end if
$s \leftarrow \sigma(\text{clip}(\text{logit}\, p, \pm 4) + 5(E^+ - E^-))$
if genuine alert or notice with no blocking cue **then**
    $s \leftarrow \min(s, 0.15)$
end if
return verdict $\mathbf{1}[s \geq 0.5]$, p, s, highlighted phrases, explanation

4.4 Data Design

4.4.1 Database

Table 4.2 lists the main tables of the PostgreSQL database.

Table 4.2: Main database tables

Table	Contents
users	Account, hashed password, role, preferences
refresh_tokens	Issued refresh tokens for rotation and revocation
messages	Each analysed input with its type (text, URL, e-mail, image, PDF, QR)
predictions	Verdict, classifier probability, risk score, category, explanation, model version
cases, case_scan_links, case_timeline_entries	Investigation cases, the scans linked to them and their notes
community_reports	One report per user for a number, link domain, UPI ID or e-mail
alerts, audit_log, model_registry_meta	Alerts, security audit trail and registered model versions

4.4.2 Datasets

ScamGuard-IN-2026 has seven attributes per row: text, label (scam or legitimate), source, subtype, is_synthetic, campaign (near-duplicate group) and split (train, development, test or 2026 test).

The **cross-domain datasets** are stored after cleaning with the attributes message, label (spam = 1, ham = 0) and domain (UCI or India). The study also writes every result table as CSV, the error-annotation sheet with a manual_category column, a results.json summary and four joblib models: Naive Bayes and Logistic Regression, each in a baseline and an adapted version.

Chapter 5

Methodology

5.1 Tools and Techniques

- **TF-IDF** weights a term by how often it occurs in a message and how rare it is across messages [18]. The main detector uses word unigrams and bigrams together with character 2–5-grams; the cross-domain study compares word unigrams and bigrams with character 3–5-grams.
- **Logistic regression** gives a linear, interpretable model whose coefficients show which words push a message towards scam.
- **Multinomial Naive Bayes** models word frequencies per class and applies Bayes' rule; it is the classical baseline of the cross-domain study.
- **MinHash with locality-sensitive hashing** [17] groups near-identical messages into campaigns.
- **Confident learning** [14] guides a second review of labels that a cross-validated model contradicts.
- **Regular-expression intent rules** detect requests for OTPs, payments to receive money, remote-access apps and legal threats, and protective signals such as "do not share your OTP".
- **Supervised domain adaptation** by pooling source data with a labelled target sample [23, 22].
- **Bootstrap confidence intervals, mean ± SD over seeds and McNemar's test** [24, 25] quantify uncertainty and compare models on the same messages.
- **scikit-learn** [19] implements the models; **ftfy** repairs broken text encodings; FastAPI, Next.js and Streamlit implement the applications.

5.2 Data Set with Explanation

5.2.1 ScamGuard-IN-2026 (main detector)

Indian scam messages and Indian genuine service messages share most of their vocabulary, and public smishing data contains only reported messages with unverified labels. ScamGuard-IN-2026 was built to provide real Indian genuine messages, checked labels and current scam types. Table 5.1 lists its sources.

Table 5.1: Sources of ScamGuard-IN-2026

Source	Licence	Genuine	Scam	Use
R1: IMC 2025 smishing collection, India subset, audited [1]	CC BY 4.0	963	3,987	train, development, test
R2: India Spam SMS collection, audited [20]	MIT	1,694	6	train, development, test
R3: 2025–26 advisory and news texts	Quoted with source	1	11	test only
S: Synthetic templates	Project	2,950	1,787	training only
Total		**5,608**	**5,791**	**11,399 messages**

Label audit. The IMC 2025 messages are messages that users *reported*. We made 2,922 audit decisions on the reported near-duplicate groups: 1,253 were confirmed as scams by rule (a strong fraud cue such as an account threat with a KYC instruction, a prize with a fee or a request to share an OTP, and no genuine cue; a spot check of 40 found no error), and 1,669 were read one by one (583 scam, 867 genuine, 219 unclear and dropped). A cross-validated model then flagged 44 rows whose label it strongly contradicted; each was re-read, giving 19 corrections and 7 removals. After the audit, 963 of 4,950 retained IMC rows (19.5%) are genuine.

Campaigns and splits. Messages are grouped into a campaign when their MinHash-estimated Jaccard similarity over word 3-shingles is at least 0.6. The 6,662 real messages form 4,085 campaigns. Each campaign goes entirely to training (60%), development (15%) or test (25%), chosen by a hash of its first message (Table 5.2).

Table 5.2: ScamGuard-IN-2026 splits

Split	Genuine	Scam	Content
Train	4,581	4,064	real and synthetic
Development	361	507	real only
Test	665	1,209	real only, unseen campaigns
2026 test	1	11	advisories

5.2.2 Datasets of the cross-domain study

Dataset A (source) is the UCI SMS Spam Collection [4] (CC BY 4.0): 5,574 UK messages, 4,827 ham and 747 spam, documented in its original paper. **Dataset B (target)** is the India Spam SMS Classification dataset [20] (MIT licence, 2024): 2,267 messages with fields Msg and Label, 1,522 ham and 745 spam, contributed through an online form without a documented labelling procedure.

Audit. Dataset B had one empty message and 204 exact duplicates; 176 rows stored emoji and punctuation as broken byte sequences, and 40 rows had Devanagari text irrecoverably replaced by question marks. No message had two different labels. A spot check of ten messages per class found plausible labels: Indian spam is mostly operator, bank and retail promotions with links, and ham is mostly short personal or group-chat messages. One message ("ok") occurred in both datasets and was removed from B.

Cleaning. The same steps were applied to both datasets: labels mapped to spam = 1 and ham = 0, invalid rows dropped, broken encodings repaired with ftfy, text lowercased, spaces normalised, and exact duplicates removed. URLs, numbers and punctuation were kept because they carry spam signals. Table 5.3 gives the result, and Figures 5.1 and 5.2 show the class and length distributions.

Table 5.3: Cross-domain datasets before and after cleaning

Dataset	Raw	Clean	Ham	Spam	Spam %
A: UCI SMS Spam Collection	5,574	5,158	4,517	641	12.4
B: India Spam SMS	2,267	2,050	1,314	736	35.9

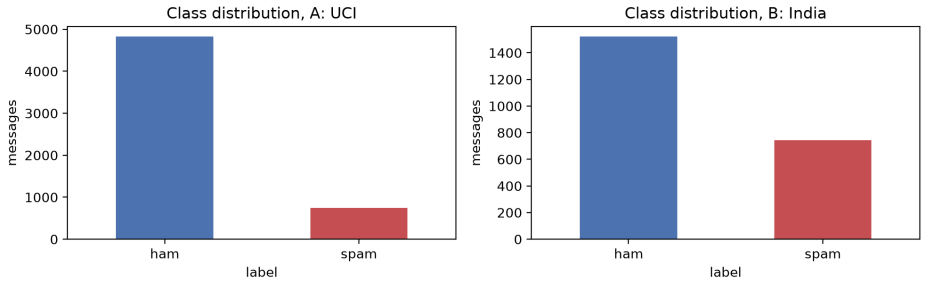


Figure 5.1: Class distribution of Dataset A (UCI) and Dataset B (India) before cleaning

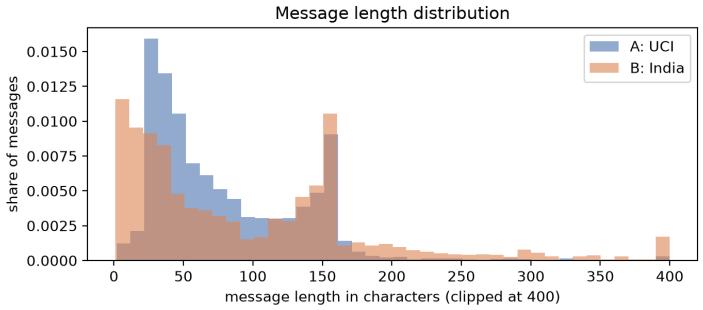


Figure 5.2: Message length distribution of the two datasets

5.3 Flowchart of Methodology from Start to End

Figure 5.3 shows the methodology of the main detector, and Figure 5.4 the methodology of the cross-domain study.

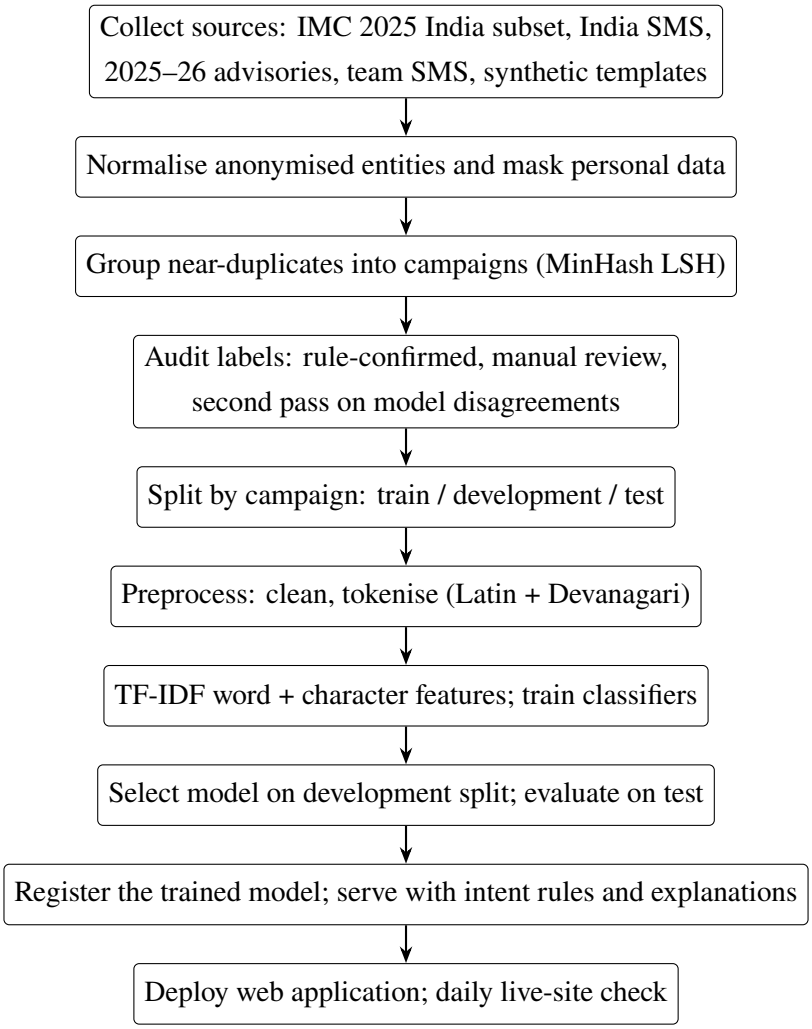


Figure 5.3: Flow of the methodology

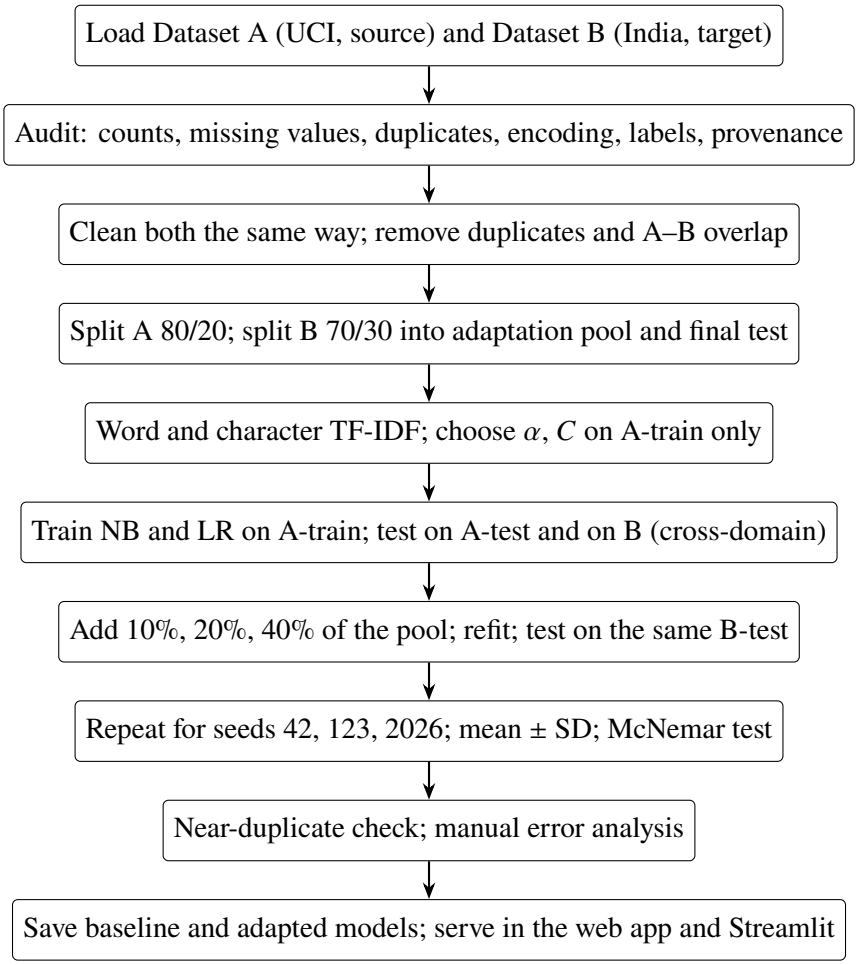


Figure 5.4: Flow of the cross-domain study

5.4 Explanation of Methodology in detail with Equations and Formulae

5.4.1 Problem definition

For a message x, the detector computes a score $s(x) \in [0, 1]$ and predicts

$$\hat{y} = \begin{cases} 1 \text{ (scam/spam)} & \text{if } s(x) \geq 0.5 \\ 0 \text{ (legitimate/ham)} & \text{otherwise.} \end{cases} \tag{5.1}$$

5.4.2 Preprocessing

In the main detector, the cleaner replaces e-mail addresses, URLs (including bare links such as bit.ly/x) and phone numbers with placeholder tokens, lowercases the text, removes punctuation while keeping Devanagari vowel signs, and replaces remaining numbers with a number token. The tokenizer keeps Latin and Devanagari words, removes English stop words and applies a light suffix stemmer. For example, *"Dear customer, your SBI account will be blocked today. Update your KYC at http://sbi-kyc.xyz and share the OTP"* becomes dear customer sbi account will block today update kyc __url__ share otp. The same code runs in training and in the live service. Because punctuation and digits are removed before feature extraction, dotted or digit-substituted spellings such as *K.Y.C* or *bl0cked* lose their keyword; such messages are caught by the other words and by the intent rules. In the cross-domain study the cleaning is lighter, as the guide specified: text is only lowercased and whitespace-normalised, so URLs, numbers and punctuation remain in the text; the word vectorizer splits on punctuation, while the character vectorizer keeps it.

5.4.3 Feature extraction

For a term t in message d, with N training messages and document frequency $\mathrm{df}(t)$, the TF-IDF weight is

$$w_{t,d} = g(\mathrm{tf}_{t,d}) \cdot \left(\ln \frac{1+N}{1+\mathrm{df}(t)} + 1 \right), \tag{5.2}$$

followed by normalising each message vector to unit length. The main detector uses sublinear term frequency, $g(\mathrm{tf}) = 1 + \ln \mathrm{tf}$, with word 1–2-grams (up to 30,000) and character 2–5-grams (up to 60,000), giving 35,148 features after pruning. The cross-domain study uses raw term frequency, $g(\mathrm{tf}) = \mathrm{tf}$, with word 1–2-grams (up to 20,000) or character 3–5-grams within word boundaries. Every vectorizer is fitted on training data only.

Worked example. For the three messages "update kyc now", "kyc update otp" and "order delivered today" ($N = 3$), the first message has df = 2 for *update* and *kyc* and df = 1 for *now*, so their IDF values are $\ln(4/3) + 1 = 1.288$ and $\ln 2 + 1 = 1.693$. After normalisation the vector is $(0.518, 0.518, 0.681)$: the rarest word receives the largest weight.

5.4.4 Logistic regression

Logistic regression estimates

$$p(x) = \sigma\big(\mathbf{w}^\top \phi(x) + b\big) = \frac{1}{1 + e^{-(\mathbf{w}^\top \phi(x) + b)}}, \tag{5.3}$$

by minimising the class-weighted, ℓ_2-regularised log-loss

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i} \Big[- y_i \ln p(x_i) - (1 - y_i) \ln \big(1 - p(x_i)\big) \Big], \tag{5.4}$$

with class weights $\alpha_c = n/(2n_c)$. The main detector uses $C = 16$; in the cross-domain study C was chosen by cross-validation on the source training data ($C = 100$ for word and $C = 10$ for character features). In the main model the bias is $b = -4.82$, and the largest weights include *will block* +2.87, *kyc* +2.69 and *prize* +2.98 towards scam, and *not share* −1.85 and *branch* −2.22 towards genuine. For the SBI message above $z = \mathbf{w}^\top \phi(x) + b = 8.99$ and $p = 0.9999$; for the genuine message "482913 is your OTP for Rs 2,499 at Amazon. Do not share it with anyone" $z = -5.53$ and $p = 0.004$.

5.4.5 Multinomial Naive Bayes

Naive Bayes predicts the class with the larger posterior,

$$\hat{c} = \arg\max_{c \in \{0,1\}} \Big[\ln P(c) + \sum_{t} x_t \ln P(t \mid c) \Big], \qquad P(t \mid c) = \frac{N_{tc} + \alpha}{N_c + \alpha |V|}, \tag{5.5}$$

where x_t is the TF-IDF value of term t, N_{tc} the total weight of t in class c, N_c the total weight of class c, $|V|$ the vocabulary size and α the smoothing. Cross-validation on the source training data chose $\alpha = 0.03$; the library default $\alpha = 1$ is reported for comparison.

5.4.6 Intent evidence and fusion

Thirty-one patterns detect 26 scam intents with hand-set weights w_i, and ten protective signals have weights v_j. With $E^+ = \sum_{i \in I(x)} w_i$ (halved when only weak context cues match) and $E^- = \sum_{j \in P(x)} v_j$ (halved when $E^+ \geq 0.5$), the final score is

$$s(x) = \sigma\Big(\operatorname{clip}\big(\operatorname{logit} p(x), -4, 4 \big) + 5\,(E^+ - E^-) \Big), \tag{5.6}$$

capped at 0.15 when the message has the shape of a genuine transaction alert or service notice without any blocking cue. A "decoy" rule catches messages that say "never share

your OTP" and then ask for it.

5.4.7 Near-duplicate grouping

Each message is reduced to the set S of its word 3-shingles after replacing links and digits. Two messages belong to the same campaign when

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|} \geq 0.6, \tag{5.7}$$

estimated with 96 MinHash permutations and locality-sensitive hashing.

5.4.8 Domain adaptation

Let A_{train} be the source training split, P the target adaptation pool and $S_f \subset P$ a stratified sample with $|S_f| = \lfloor f\,|P| \rfloor$. The adapted model is trained on

$$D_f = A_{train} \cup S_f, \qquad f \in \{0, 0.1, 0.2, 0.4\}, \tag{5.8}$$

with the vectorizer refitted on D_f, so that target words enter the vocabulary. With $|P| = 1{,}435$, the samples contain 143, 287 and 574 messages. Every f is scored on the same final target test set B_{test} (615 messages).

5.4.9 Evaluation metrics

With scam or spam as the positive class:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \tag{5.9}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{5.10}$$

$$\text{False-alarm rate} = \frac{FP}{FP + TN}. \tag{5.11}$$

Confidence intervals come from 1,000 bootstrap resamples, and results over seeds are reported as mean ± standard deviation.

5.4.10 Statistical test

To compare two models on the same test messages, let b be the number of messages only the first model classifies correctly and c the number only the second classifies correctly. Under the null hypothesis that both models are equally accurate, $\min(b, c)$ follows a binomial distribution with $n = b + c$ and probability 0.5; the exact McNemar

test [24, 25] gives the two-sided p-value.

Chapter 6

Results and Discussion

6.1 Presentation of Results

6.1.1 Experimental setup and testing

For the main detector, models were selected on the development split and then trained on the training and development splits together (9,513 messages). Results are on the real test split: 1,874 messages (665 genuine, 1,209 scams) from campaigns never seen in training. A 259-message challenge set that the team wrote while designing the rules is reported separately as a development check, not as an independent test. For the cross-domain study, all settings were chosen by cross-validation on the UCI training split, and every adapted model was scored on the same untouched Indian test set; the study was repeated with seeds 42, 123 and 2026. Experiments ran on a two-core CPU with Python 3.13 and scikit-learn 1.7.2.

The software was tested at three levels: 836 automated back-end and ML tests (preprocessing, features, registry, detector, rules, cross-domain service and every API route, including a test that the served cross-domain models reproduce the notebook's scores exactly) and 14 front-end tests, all passing; continuous integration on every change before deployment; and checks of the live site after each deployment.

6.1.2 Main detector on ScamGuard-IN-2026

Table 6.1: Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate)

System	Acc.	Prec.	Rec.	F1	FAR	AUC
Classifier only	97.5	98.6	97.6	98.1	2.6	0.997
Deployed system (hand rules)	97.1	98.2	97.2	97.7	3.2	0.995
Learned fusion	**97.8**	**98.7**	**97.8**	**98.3**	**2.3**	0.997

The deployed system catches 1,175 of 1,209 test scams and passes 644 of 665 genuine messages. The 95% confidence interval of its accuracy is 96.3–97.8%, and of its false-

alarm rate 2.0–4.6%. On the twelve 2025–26 advisory messages it detects 10 of 11 scams and passes the genuine notice; this set is too small for a firm conclusion.

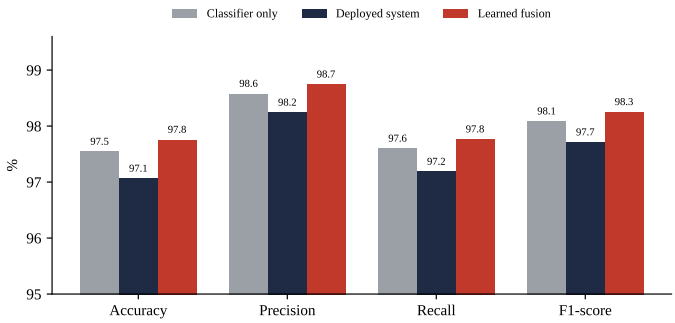


Figure 6.1: Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%)

Models and features. Table 6.2 compares classifiers on the same split. Logistic regression with word and character features has the lowest false-alarm rate among the strong models. Deep baselines trained under the same protocol did not do better: a character-level CNN reached an F1 of 97.3% with 4.4% false alarms and a word-level BiLSTM 97.4% with 5.6%, against 97.8% and 3.0% for logistic regression under that protocol; the differences were not significant (McNemar $p = 0.09$ and 0.11).

Table 6.2: Classifiers on the real test set (%); "Challenge" is F1 on the 259 team-written messages

Model	Features	F1	False alarms	Challenge F1
Naive Bayes	word	96.2	6.2	88.0
Naive Bayes	word + char	95.5	6.5	86.6
Logistic regression	word	98.0	2.7	89.9
Logistic regression	char	96.9	4.8	86.8
Logistic regression	**word + char**	**98.1**	**2.6**	90.2
Linear SVM	word	97.7	5.6	91.6
Linear SVM	word + char	98.2	3.9	92.0
Random forest	word	97.5	2.7	73.7
Random forest	word + char	97.6	3.3	73.9

Label audit. Training the same model with every reported message labelled as a scam gives 99.4% recall but flags 34.0% of genuine messages; with audited labels the false-alarm rate is 2.6% at 97.6% recall (210 against 23 discordant messages, McNemar $p < 10^{-30}$, Figure 6.2).

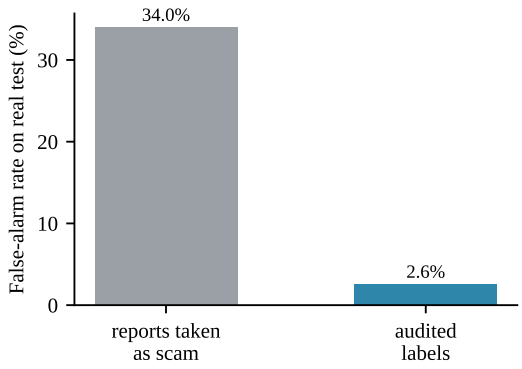


Figure 6.2: False-alarm rate with report labels and with audited labels

Split and synthetic data. With a random split of the real data, 793 test messages had a near-duplicate in training; the random split gave an F1 of 98.3% and a 0.8% false-alarm rate, the campaign-aware split 97.7% and 1.4%, so the random split understates false alarms. Table 6.3 varies the training data. Synthetic messages alone do not resemble real Indian SMS; real messages alone recognise only 35.6% of the scams in the team-written challenge set; together they keep real-data F1 at 98.1% and raise challenge recall to 82.2%, at the cost of a higher false-alarm rate on real data (2.6% against 1.4%).

Table 6.3: Effect of the training data (logistic regression, word + char, %)

Training data	Real F1	Real FAR	Challenge recall	Challenge F1
Synthetic only	87.4	31.1	86.4	90.3
Real only	97.7	1.4	35.6	51.2
Real + synthetic	98.1	2.6	82.2	90.2

Intent rules. On real messages from unseen campaigns the hand-weighted rules lower F1 from 98.1% to 97.7% and raise false alarms from 2.6% to 3.2%; on the team's own challenge messages they raise F1 from 90.2% to 95.6%. Learned fusion weights are significantly better than the hand-set ones on real data ($p = 0.002$). The rules are kept for phrase-level explanations and for scam types absent from the real training data, not as an accuracy gain.

Operating point and errors. The deployed score has a ROC-AUC of 0.995 (Figures 6.3 and 6.4). At the default threshold it misses 34 scams and flags 21 genuine messages, 15 of which are reported-but-genuine bank messages that ask the user to click a link. A threshold of 0.9 lowers false alarms to 1.1% at 92.7% recall. Recall is 97.7% on banking scams (895 of 916). The model and vectorizer take 1.6 MB, training takes 2.7 seconds, and scoring one message takes about 2 ms (4 ms with the rules).

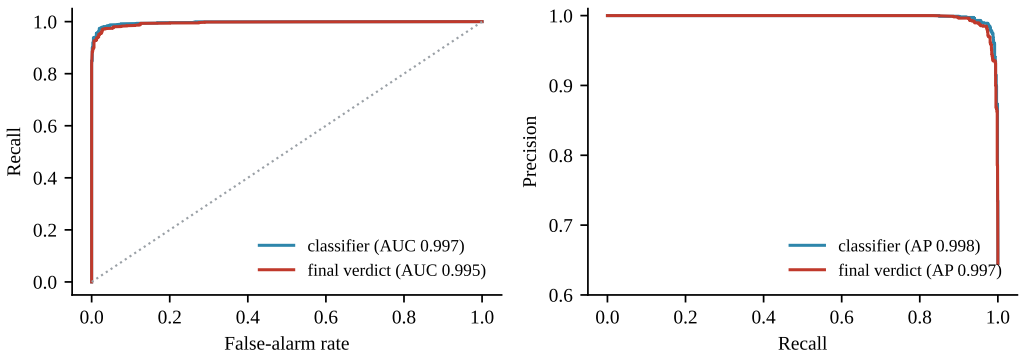


Figure 6.3: ROC and precision–recall curves on the real test set

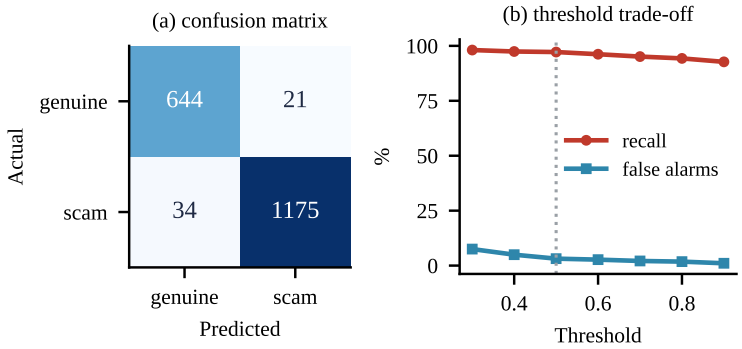


Figure 6.4: (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold

6.1.3 Cross-domain generalization (RQ1 and RQ2)

Table 6.4 applies the models trained on UCI only (seed 42) to all 2,050 cleaned Indian messages without retraining. Every model loses between 0.23 and 0.36 F1. Precision stays high (0.88–0.96) but recall falls to 0.40–0.55: the models rarely raise false alarms but miss about half of the Indian spam. Logistic Regression with word features transfers best.

Table 6.4: Trained on Dataset A only (seed 42): spam F1 on the UCI test split and on all of Dataset B

Model	Feat.	F1 UCI	F1 India	Prec. India	Rec. India
Naive Bayes	word	0.906	0.551	0.883	0.401
Logistic Regression	word	0.911	**0.678**	0.931	0.533
Linear SVM	word	0.914	0.615	0.949	0.455
Naive Bayes	char	0.948	0.684	0.908	0.549
Logistic Regression	char	0.940	0.638	0.942	0.482
Linear SVM	char	0.943	0.578	0.956	0.414

The smoothing of Naive Bayes is decisive under domain shift: with the library default $\alpha = 1$, Naive Bayes reached F1 0.726 on UCI but only 0.014 on Indian messages, labelling almost every Indian message as ham; with α chosen on source data alone these became 0.906 and 0.551.

Why the domains differ. On average 20.7% of the words in an Indian message never occur in the UCI training split, against 7.8% for held-out UCI messages. Indian messages are longer (median 81 against 59 characters), 87.0% of Indian spam contains a URL against 1.6% of UCI test messages, and 24.3% of Indian spam contains a currency marker such as “Rs” or “INR”.

6.1.4 Lightweight domain adaptation (RQ4)

Table 6.5 and Figure 6.5 show the effect of adding labelled Indian messages. Most of the gain comes from the first 10% of the pool: 143 messages raise mean target F1 from 0.681 to 0.964 for Logistic Regression and from 0.582 to 0.950 for Naive Bayes. Source-domain F1 stays near 0.93. The gain was positive in all three seeds for every model, feature set and fraction, and McNemar’s test gave $p < 0.05$ in all 36 comparisons. Adaptation mainly converts missed spam into detected spam: for Logistic Regression at seed 42, false negatives fall from 96 to 3 while false positives stay similar (10 and 9, Figure 6.6).

Table 6.5: Spam F1 on the final Indian test set (615 messages), word TF-IDF, mean ± SD over three seeds

Indian data added	Naive Bayes	Logistic Regression
0% (source-only)	0.582 ± 0.018	0.681 ± 0.025
10% (143 messages)	0.950 ± 0.006	0.964 ± 0.004
20% (287 messages)	0.967 ± 0.007	0.968 ± 0.005
40% (574 messages)	**0.978** ± 0.009	**0.975** ± 0.005
100% (reference only)	0.987 ± 0.007	0.987 ± 0.003

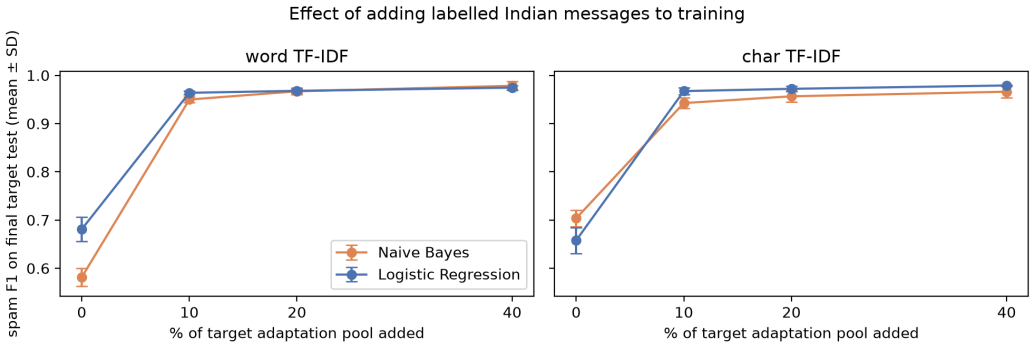


Figure 6.5: Spam F1 on the final Indian test set as labelled Indian messages are added (mean ± SD over three seeds)

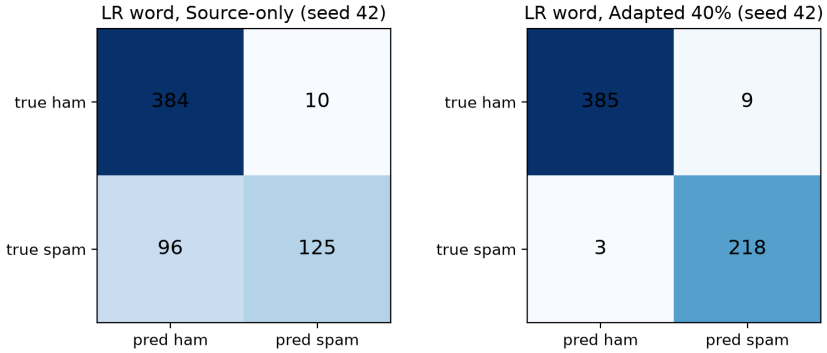


Figure 6.6: Logistic Regression (word TF-IDF, seed 42) on the Indian test set, before and after adding 40% of the pool

Near-duplicate check. Indian promotional SMS are templated, so a random split can place a near-copy of a test message in the adaptation pool. Grouping Dataset B with MinHash (Jaccard ≥ 0.6) gave 1,811 groups for 2,050 messages; under the random split 86–104 of the 615 test messages had a near-copy in the pool. Repeating adaptation with a group-aware split reduced the gains only slightly (Table 6.6), so the adaptation effect is not explained by memorised templates.

Table 6.6: Target F1 with a random and a group-aware split of Dataset B (word TF-IDF, mean ± SD)

Model	Added	Random split	Group-aware split
Logistic Regression	0%	0.681 ± 0.025	0.652 ± 0.062
Logistic Regression	10%	0.964 ± 0.004	0.952 ± 0.008
Logistic Regression	40%	0.975 ± 0.005	0.969 ± 0.011
Naive Bayes	0%	0.582 ± 0.018	0.551 ± 0.017
Naive Bayes	10%	0.950 ± 0.006	0.937 ± 0.015
Naive Bayes	40%	0.978 ± 0.009	0.960 ± 0.015

6.1.5 Word versus character features (RQ3)

Neither representation won for both classifiers (Table 6.7, Figure 6.7). Character n-grams improved source-only target F1 for Naive Bayes but lowered it slightly for Logistic Regression; after adaptation the two are within 0.01 for Logistic Regression.

Table 6.7: Source-only models: word versus character TF-IDF (mean ± SD over three seeds)

Model	Features	Source F1	Target F1	Target recall
Naive Bayes	word	0.927	0.582 ± 0.018	0.430
Naive Bayes	character	0.941	0.704 ± 0.017	0.581
Logistic Regression	word	0.928	0.681 ± 0.025	0.535
Logistic Regression	character	0.942	0.658 ± 0.027	0.502

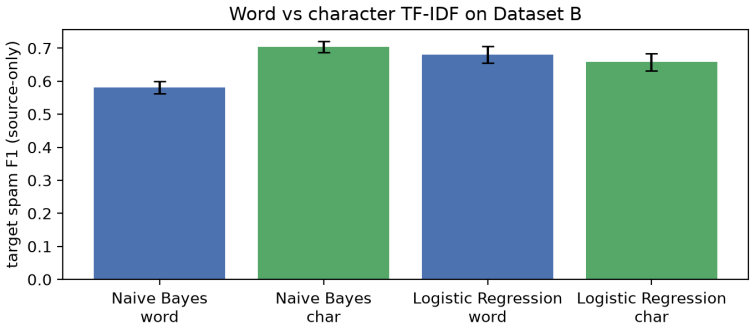


Figure 6.7: Target F1 of the source-only models with word and character TF-IDF

6.1.6 Error analysis (RQ5)

All 118 errors of Logistic Regression (word TF-IDF, seed 42) on the Indian test set were read individually and categorised (Table 6.8, Figure 6.8). Before adaptation, most errors are Indian promotions whose vocabulary the UCI model never saw. After adaptation only 12 errors remain, eight of them very short chat messages with almost no content. Nine messages carry questionable labels in Dataset B.

Table 6.8: Error categories on the Indian test set (Logistic Regression, word TF-IDF, seed 42)

Category	Source-only	Adapted (40%)
Indian brand or telecom promotion with unfamiliar vocabulary	61	0
Operator service notice (data alert, pack expiry, missed call)	14	0
Questionable label: safety advice or receipt labelled spam	9	1
Very short or context-free chat (ham)	8	8
Fraud with disguised spelling (“t0”, “n0w”)	6	2
Loan or debt message	3	0
Market or work commentary with spam-like words (ham)	2	1
Code-mixed Hinglish promotion	2	0
Lost characters (“????”)	1	0
Total	**106**	**12**

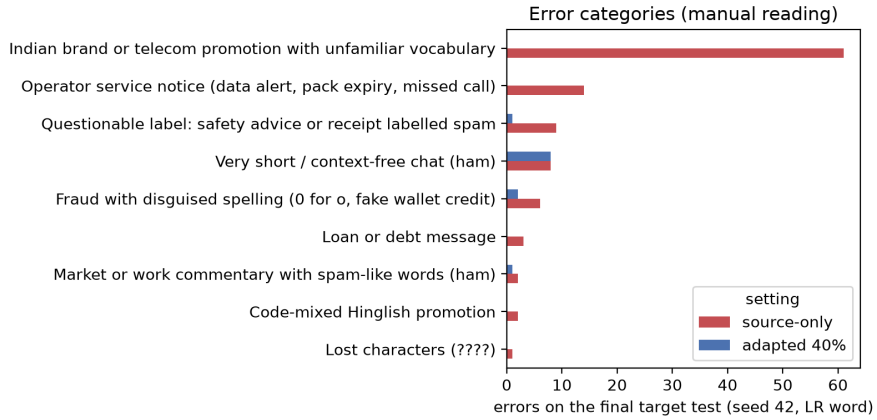


Figure 6.8: Error categories before and after adaptation

6.1.7 Application

The deployed application implements every functional requirement in Section 3.2. The cross-domain page (Appendix A) lets a user classify the same message with the baseline and the adapted model: an Indian retail offer that the baseline Logistic Regression scores 0.149 (ham) is scored 0.977 (spam) by the adapted model. Selected screens are shown in Appendix A.

6.2 Limitations of the study

- **Time coverage.** The reported messages in ScamGuard-IN-2026 are mostly from 2022–2024; 2025–26 scam types appear in training only through synthetic messages and in testing through 12 advisory messages.
- **Label reliability.** One annotator made the manual audit decisions, so no inter-annotator agreement is available; the Indian dataset of the cross-domain study has no documented labelling procedure, and nine of its labels look doubtful.
- **Two domains only.** The cross-domain study uses one source and one target dataset and cannot claim robustness across domains in general; three seeds give a limited estimate of variability.
- **Shortcut in Dataset B.** None of its ham messages contains a URL while 87% of its spam does, so an adapted model can partly rely on that cue.
- **Obfuscation.** The main detector's cleaner removes dotted and digit-substituted spellings before feature extraction; the intent rules catch many of these, but adaptive attacks were not tested.
- **Scope.** Single messages only; English and Hindi only; in the cross-domain study "spam" includes legitimate promotions.

6.3 Conclusion

Three results stand out. First, label quality mattered more than model choice: report labels gave 34.0% false alarms and audited labels 2.6%, while every reasonable classifier, including CNN and BiLSTM baselines, scored within a few points of the others. Second, honest evaluation changes the picture: a campaign-aware split exposes errors that a random split hides, and rules that look strong on the developers' own messages do not add accuracy on independent data. Third, conventional spam classifiers do not transfer across countries: trained on UK messages they miss about half of Indian spam, but as few as 143 labelled Indian messages restore an F1 above 0.95 without harming source

performance, consistently across seeds and significantly by McNemar's test.

Chapter 7

Conclusion

ScamGuard detects scam and spam SMS for Indian users and explains every verdict. Built on the audited ScamGuard-IN-2026 corpus with a campaign-aware split, the deployed detector reaches 97.1% accuracy with a 3.2% false-alarm rate on real messages from unseen campaigns, answers in milliseconds and shows the exact phrases behind each decision. The cross-domain study proposed by our guide shows that the same kind of classifier, when trained on another country's messages, loses about a third of its F1 on Indian SMS, and that a small, carefully labelled local sample is enough to recover it. The practical lesson for anyone deploying such a filter is to measure it on local data and to spend effort on a few hundred verified local labels before reaching for a more complex model.

Both results are delivered in a working, deployed application with a quick check, full analysis, Hindi support, safety tools, a cross-domain page and a Streamlit demonstration, and every number in this report can be regenerated from the project repository.

Future work.

- Complete the team-contributed 2026 test set and repeat both evaluations on it.
- Add a second annotator for the label audit and report agreement.
- Extend the cross-domain study to more target datasets and seeds, and to adaptation with fewer than 10% of the pool.
- Normalise obfuscated spellings before feature extraction and evaluate against adaptive and paraphrased attacks.
- Extend to Kannada, Tamil, Telugu and other Indian languages, and compare with small transformer models under the same protocols.

References

[1] S. Agarwal, A. Papasavva, G. Suarez-Tangil, and M. Vasek, "Fishing for smishing: Understanding SMS phishing infrastructure and strategies by mining public user reports," in *Proc. ACM Internet Measurement Conference (IMC '25)*, Madison, WI, USA, 2025, doi: 10.1145/3730567.3764431.

[2] S. Agarwal, G. Suarez-Tangil, and M. Vasek, "An overview of 7726 user reports: Uncovering SMS scams and scammer strategies," in *Proc. Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2026.

[3] M. M. A. Pritom, S. M. Sanjari, M. Mia, A. M. Shibli, S. M. M. Hossain, M. Ismail, and S. Xu, "Short message service (SMS) phishing attacks and defenses: A systematic review," arXiv:2604.11429, 2026.

[4] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symposium on Document Engineering*, Mountain View, CA, USA, 2011, pp. 259–262.

[5] T. A. Almeida, J. M. Gómez Hidalgo, and T. P. Silva, "Towards SMS spam filtering: Results under a new dataset," *International Journal of Information Security Science*, vol. 2, no. 1, pp. 1–18, 2013.

[6] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep learning to filter SMS spam," *Future Generation Computer Systems*, vol. 102, pp. 524–533, 2020.

[7] S. Mishra and D. Soni, "Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis," *Future Generation Computer Systems*, 2020.

[8] G. Sonowal and K. S. Kuppusamy, "SmiDCA: An anti-smishing model with machine learning approach," *The Computer Journal*, vol. 61, no. 8, pp. 1143–1157, 2018, doi: 10.1093/comjnl/bxy039.

[9] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "Machine learning driven smishing detection framework for mobile security," arXiv:2412.09641, 2024.

[10] D. Timko and M. L. Rahman, "Smishing Dataset I: Phishing SMS dataset from Smishtank.com," in *Proc. 14th ACM Conference on Data and Appli-*

cation Security and Privacy (CODASPY '24), Porto, Portugal, 2024, doi: 10.1145/3626232.3653282.

[11] F. D. M. A. Ali, S. M. Saide, R. Sousa-Silva, and H. Lopes Cardoso, "MOZ-Smishing: A benchmark dataset for detecting mobile money frauds," in *Proc. 6th Workshop on African Natural Language Processing*, Vienna, Austria, 2025, pp. 158–166, doi: 10.18653/v1/2025.africanlp-1.23.

[12] M. Labonne and S. Moran, "Spam-T5: Benchmarking large language models for few-shot email spam detection," arXiv:2304.01238, 2023.

[13] M. Ahmadi, M. Khajavi, A. Varmaghani, A. Ala, K. Danesh, and D. Javaheri, "Leveraging large language models for cybersecurity: Enhancing SMS spam detection with robust and context-aware text classification," arXiv:2502.11014, 2025.

[14] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021, doi: 10.1613/jair.1.12125.

[15] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. 60th Annual Meeting of the ACL*, Dublin, Ireland, 2022, pp. 8424–8445, doi: 10.18653/v1/2022.acl-long.577.

[16] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 2023, doi: 10.1016/j.patter.2023.100804.

[17] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of SEQUENCES 1997*, Positano, Italy, 1997, doi: 10.1109/SEQUEN.1997.666900.

[18] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] JuniorAlive, "India Spam SMS Classification," GitHub repository, 2024. [Online]. Available: https://github.com/junioralive/india-spam-sms-classification

[21] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Machine Learning*, vol. 79, pp. 151–175, 2010, doi: 10.1007/s10994-009-5152-4.

[22] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

[23] H. Daumé III, "Frustratingly easy domain adaptation," in *Proc. 45th Annual Meeting of the Association for Computational Linguistics*, Prague, Czech Republic, 2007, pp. 256–263.

[24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[25] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, doi: 10.1162/089976698300017197.

Plagiarism

The plagiarism (similarity) report and the AI-content report of this document, generated with the tool prescribed by the department, are attached on the following page.

Appendix A: Snapshots

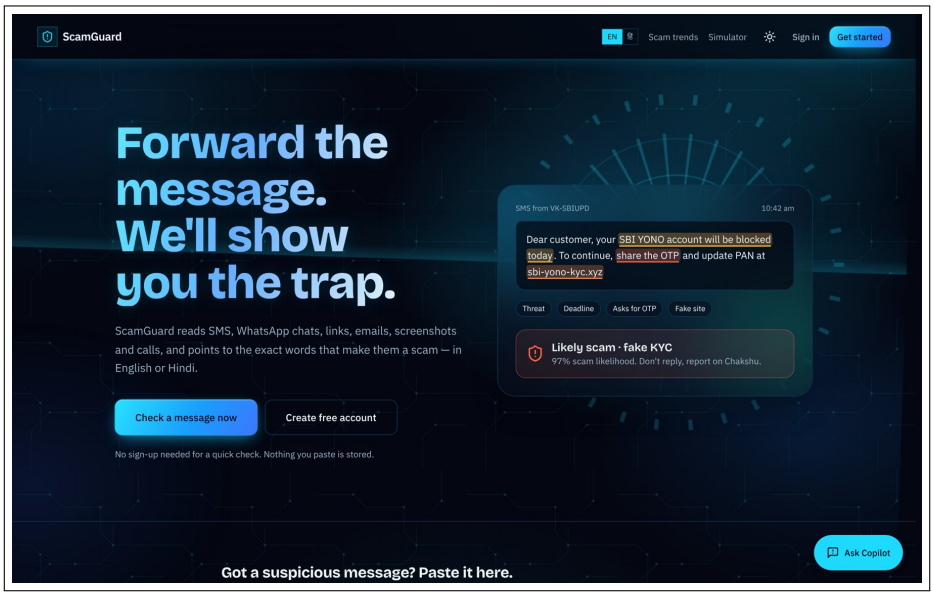


Figure A.1: Home page with the quick check

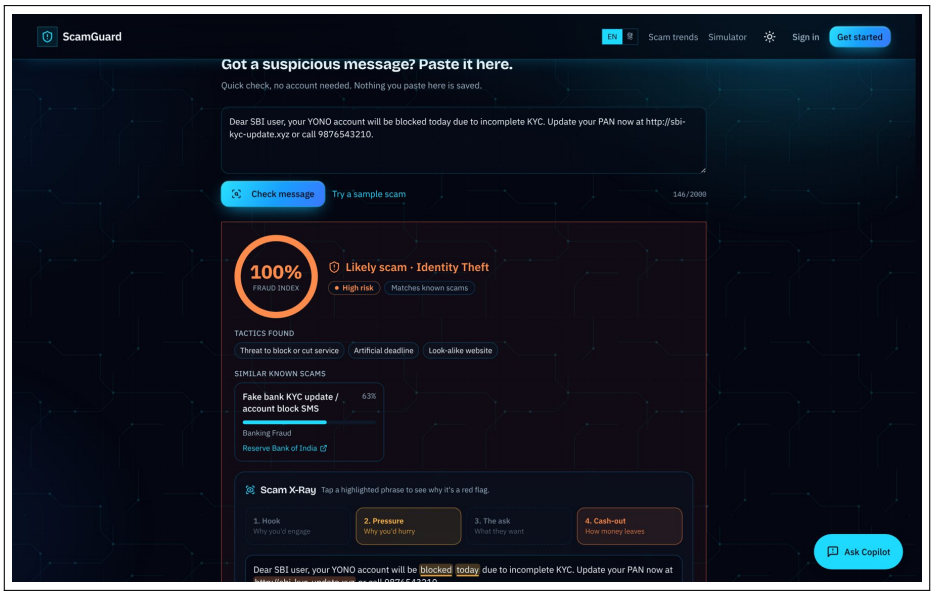


Figure A.2: Quick check of a fake KYC message: verdict, matched tactics, known pattern and Scam X-Ray

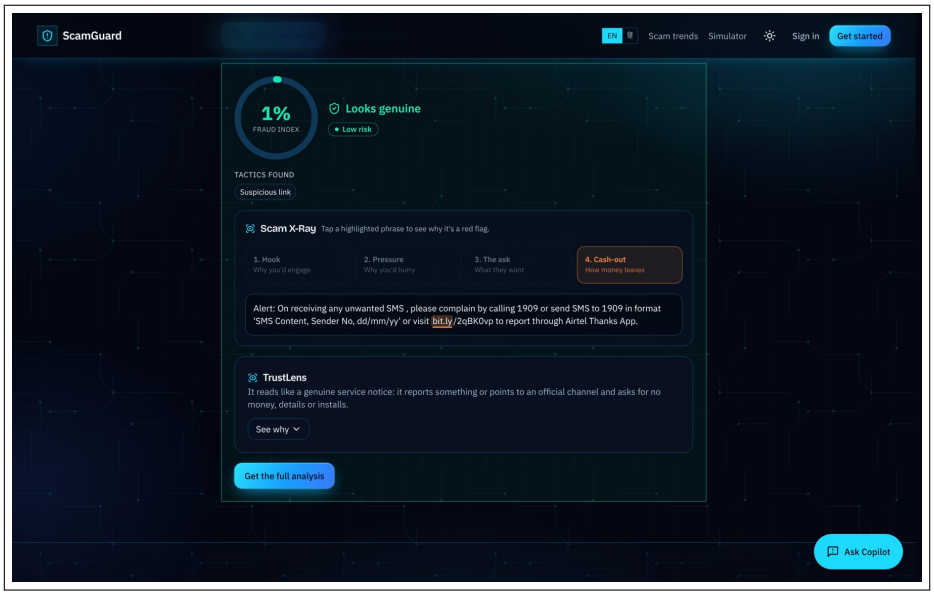


Figure A.3: Quick check of a genuine operator notice about the 1909 helpline: "Looks genuine" with the TrustLens reason

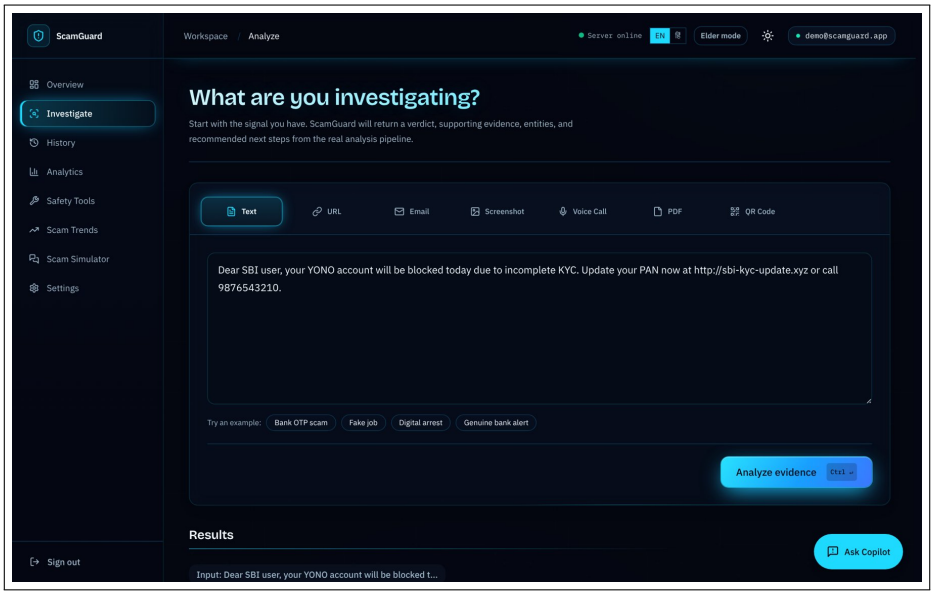


Figure A.4: Full analysis of a scam message for a signed-in user

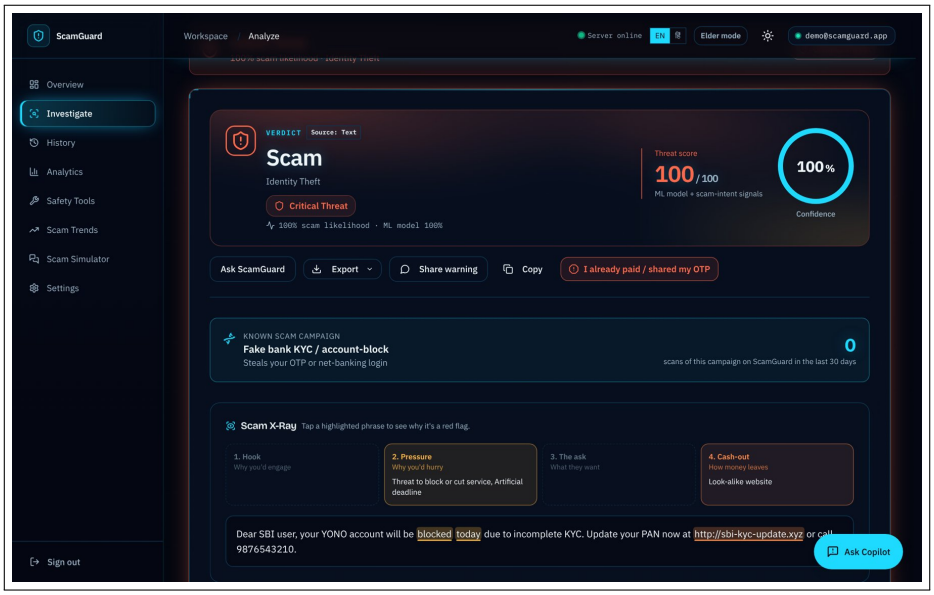


Figure A.5: Scam X-Ray: each risky phrase placed in the hook, pressure, request and cash-out stages

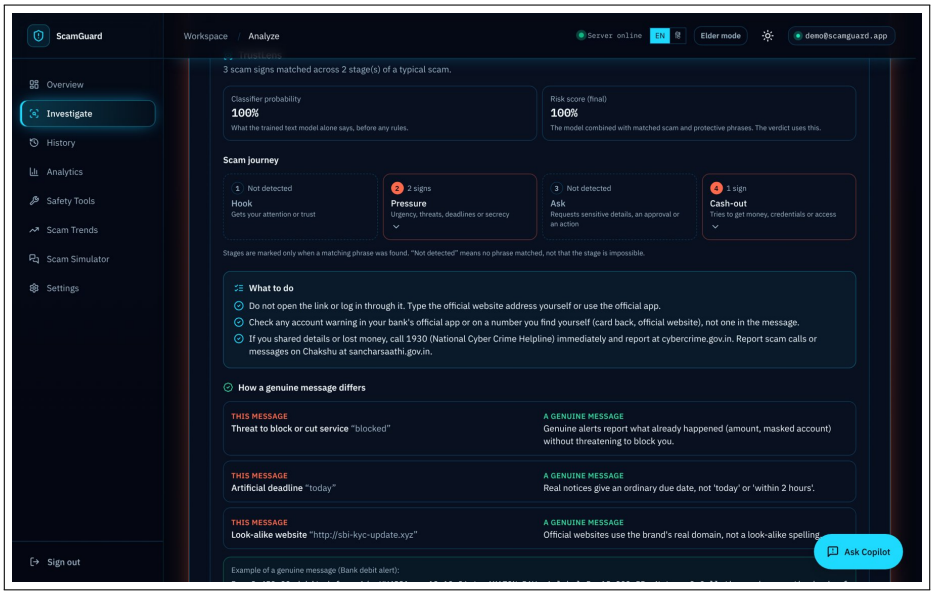


Figure A.6: TrustLens: scam journey, recommended actions and how a genuine message differs

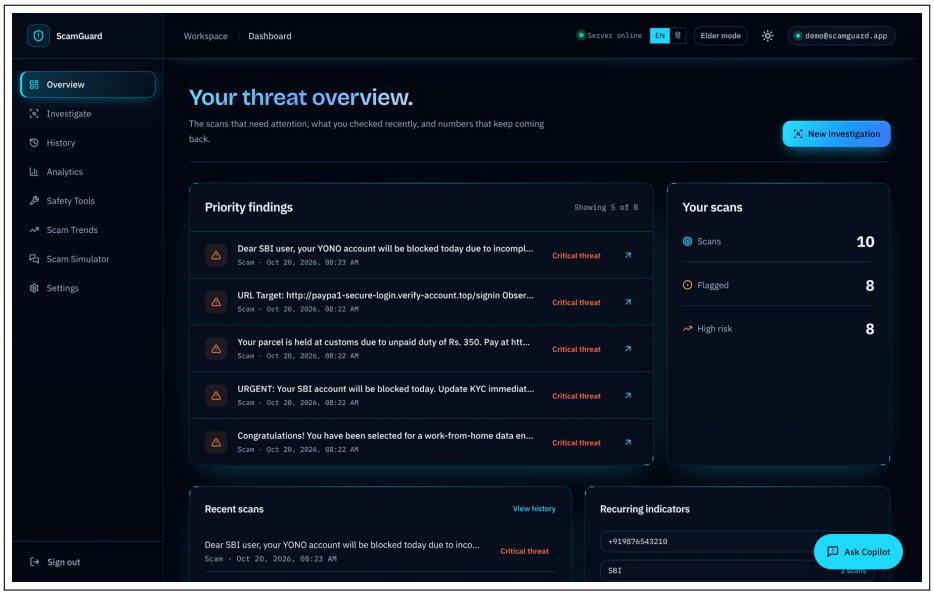


Figure A.7: Dashboard with priority findings and recent scans

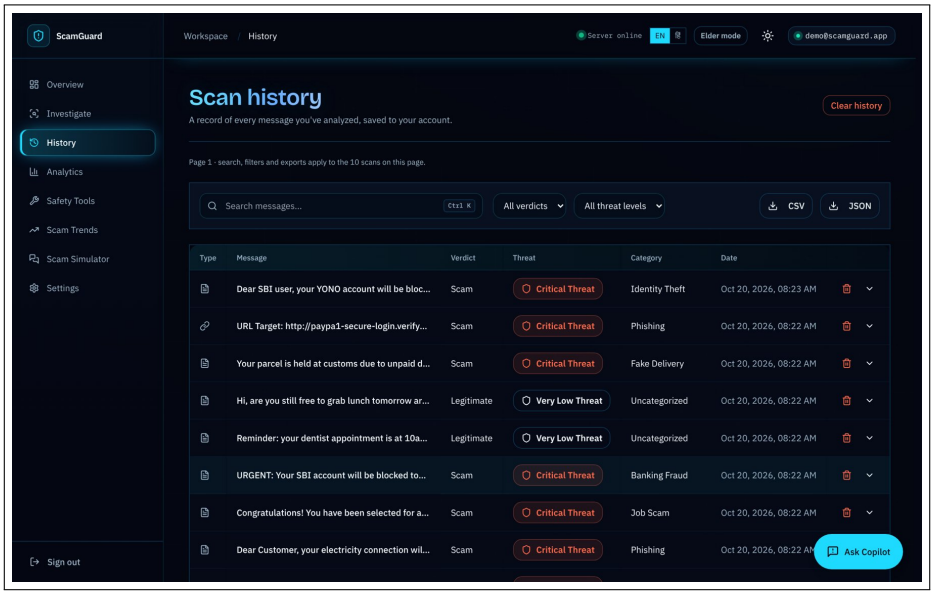


Figure A.8: Scan history with search, filters and export



Figure A.9: Analytics: verdict share, distribution and score trend

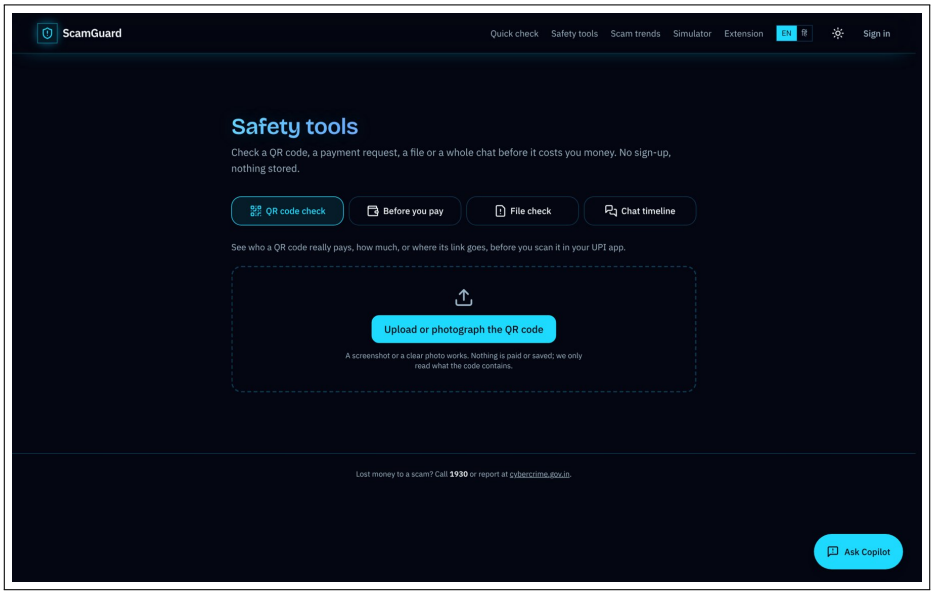


Figure A.10: Safety tools: QR code check, before-you-pay, file check and chat timeline

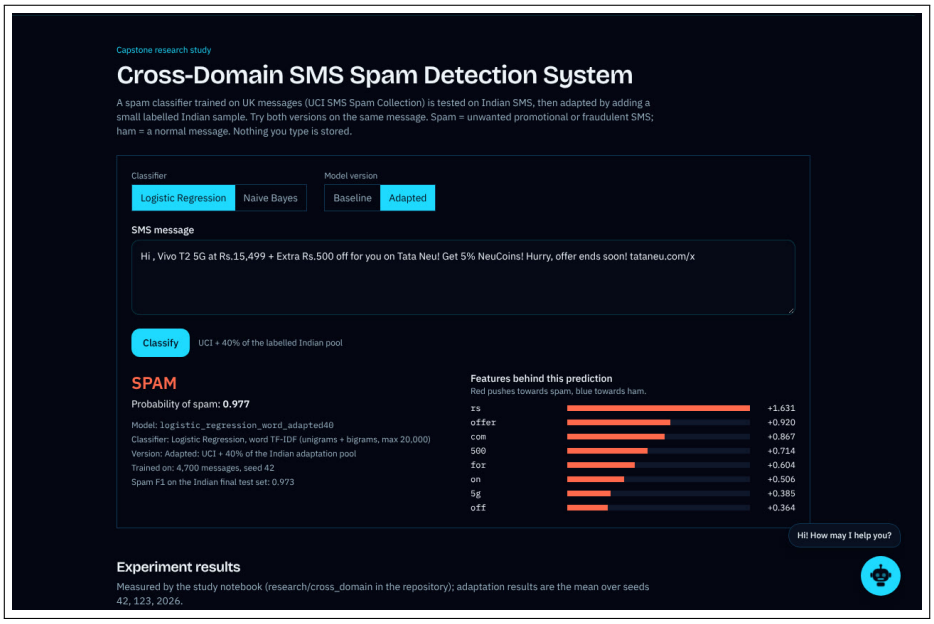


Figure A.11: Cross-domain study page: the adapted model flags an Indian promotion that the baseline misses, with the measured results

Appendix B: Details of list of publications related to this project

Author Names: Kayarvizhy N, Priyam Thakur, Bhavya Goyal, Samir Chaudhary, Sana Subodh

Paper Titles: (1) ScamGuard: AI-Based Online Scam and Fraud Detection System; (2) Cross-Domain Generalization and Lightweight Domain Adaptation for SMS Spam Detection

Name of the Conference or Journal: To be decided (manuscripts prepared in IEEE conference format, awaiting the guide's approval)

Place of the conference: –

Date of Conference: –

Appendix C: Details of Patent

No patent has been filed for this project.

Appendix D: Details of Funding

The project received no external funding. It was built with free cloud tiers and open-source software.

Appendix E: POs and PSOs Mapped

Tables E.1 and E.2 give the course outcomes of Major Project Phase-2 (23CS7PWPP2) and their CO–PO–PSO mapping; the tables that follow justify the level of each PO and PSO addressed by this project.

Table E.1: Course outcomes and where they are addressed

CO	Course outcome	Addressed in this work
CO1	Apply software engineering principles in planning, design and requirements within legal, societal and environmental constraints	Section 2.3 and Chapters 3 and 4: problem statement, requirements, architecture and modules; licensed data, masked personal data
CO2	Apply mathematics, science and engineering to implement the project and draw conclusions with integrity	Section 5.4: mathematical model; Chapter 6: results with confidence intervals and significance tests
CO3	Monitor and manage schedules, resources and work assignments	Section 1.8 and Section 3.6: work plan and constraints
CO4	Test and defend the performance of the implemented project	Chapter 6: testing, metrics and validation
CO5	Work individually and as a team	Shared work on data, model, back end and front end
CO6	Communicate, present and prepare technical documents	This report, the research paper and the presentation

Table E.2: CO–PO–PSO mapping of Major Project Phase-2

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1		2	2			2	2					2	2		
CO2	3			3	3			3						3	3
CO3											3				
CO4							1								1
CO5									1			1			
CO6										2		2			

Table E.3: Programme Outcomes Mapping with Project

PROGRAMME OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PO1	3	Applies machine learning, NLP and statistics to classify scam messages and to evaluate the classifier.
PO2	3	Analyses why existing filters fail on Indian SMS and traces the failure to dataset labels and evaluation leakage.
PO3	3	Designs a dataset, a detector and a web application that meet accuracy, latency and explanation requirements.
PO4	3	Conducts controlled experiments with bootstrap intervals, three seeds and significance tests on campaign-held-out and cross-domain test sets.
PO5	3	Uses scikit-learn, FastAPI, Next.js, Streamlit, MinHash, GitHub Actions and cloud deployment.
PO6	3	Protects users from financial fraud and points them to official reporting channels.
PO7	1	Runs on free, low-power infrastructure; the detector needs no GPU.
PO8	3	Masks personal data, respects dataset licences and reports limitations honestly.
PO9	3	Built by a team of four with shared responsibility for data, model, back end and front end.
PO10	3	Communicates results through a report, a research paper, a datasheet and an explanatory user interface.
PO11	2	Plans the work in phases and keeps the deployment within free-tier budgets.
PO12	2	Designs the dataset build so that new scam types and the 2026 test set can be added and re-evaluated.

Table E.4: Programme Specific Outcomes Mapping with Project

PROGRAMME SPECIFIC OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PSO1	2	Uses web APIs, cloud hosting, rate limiting and secure token-based communication.
PSO2	3	Applies software engineering practice: versioned models, automated tests, CI and reproducible builds.
PSO3	3	Integrates machine learning, NLP and information systems into a deployed, usable product.

Appendix F: Similarity report, AI Generated report

To be attached after the department's similarity and AI-content check.